

JAVA & SPRING BOOT

Java & Spring Boot

100 Interview Questions

QUESTIONS & ANSWERS

From syntax to Spring — the 100 that matter

This guide walks the full arc of a modern Java interview: the language core, the JVM and concurrency internals that separate mid from senior, and the Spring & Spring Boot ecosystem that runs most of the enterprise world. Every question is one a real interviewer asks, answered the way a strong candidate would explain it — the *why*, not just the definition.

Answers include runnable snippets and the trade-offs behind each choice. Work through it once cover-to-cover, then use it as a night-before refresher.

HOW TO USE THIS GUIDE

- **Read the answer, then close the page and say it out loud.** Interviews reward explanation, not recognition.
- **Type every code snippet.** Muscle memory on streams, generics, and Spring annotations pays off under pressure.
- **Focus your last hour** on the concurrency, JVM, and Spring Boot auto-configuration sections — the classic differentiators.

What's inside

Part I	Java Core — OOP, types, collections, generics, exceptions, equals/hashCode	Q1–20
Part II	Java Advanced — concurrency, threads, memory model, JVM & GC, streams, functional, records	Q21–40
Part III	Spring Core — IoC/DI, beans, scopes, configuration, AOP, bean lifecycle	Q41–60
Part IV	Spring Boot — auto-configuration, starters, properties, profiles, actuator, packaging	Q61–80
Part V	Data, Web & Security — Spring Data JPA, REST, exception handling, Spring Security, testing, microservices	Q81–100

1 The Four Pillars of OOP

OOP

CORE

Q What are the four pillars of object-oriented programming, and what does each one buy you?

A The four pillars are **encapsulation**, **inheritance**, **polymorphism**, and **abstraction**. They are not academic labels; each solves a concrete engineering problem around managing complexity and change.

Encapsulation bundles state and the behaviour that operates on it into one unit and hides the internal representation behind an API. You expose `getBalance()`, not the `double balance` field, so you can later add overdraft rules, logging, or change the field type without breaking callers. It protects invariants: nobody can set a negative balance behind your back.

Abstraction is about exposing *what* something does while hiding *how*. A `List` tells you "ordered collection you can add to"; whether it is backed by an array or a linked structure is irrelevant to the caller. Encapsulation is the mechanism; abstraction is the design goal it serves.

Inheritance models an "is-a" relationship and lets a subclass reuse and specialise a superclass. **Polymorphism** lets one reference type behave in many ways: a `Shape` reference calling `area()` dispatches to `Circle` or `Square` at runtime (dynamic dispatch). Together they let you write code against abstractions and add new types without touching existing logic.

```
abstract class Shape { abstract double area(); }
class Circle extends Shape {
    private final double r; // encapsulated state
    Circle(double r) { this.r = r; }
    @Override double area() { return Math.PI * r * r; }
}
Shape s = new Circle(2); // polymorphism: static type Shape, dynamic type Circle
```

GOTCHA

Interviewers often ask "inheritance vs composition." Inheritance is tight coupling — prefer composition ("has-a") when there is no true "is-a" relationship. Overusing inheritance leads to fragile base classes.

2 Abstract Class vs Interface

OOP

CORE

Q What is the difference between an abstract class and an interface, and what changed with default methods in Java 8?

A Both define a contract with unimplemented behaviour, but they answer different design questions. An **abstract class** models an "is-a" hierarchy with shared state and partial implementation; an **interface** models a capability ("can-do") that unrelated types can implement.

Classic differences: an abstract class can have instance fields, constructors, and any access modifier on members; a class can extend only **one** abstract class (single inheritance) but implement **many** interfaces. Interface fields are implicitly **public static final** constants, and its abstract methods are implicitly **public**.

Java 8 blurred the line by adding **default methods** (concrete methods in interfaces) and **static methods**. This enabled the JDK to add methods like **stream()** to **Collection** without breaking every existing implementer — the killer motivation for the feature. Java 9 then allowed **private** interface methods for sharing helper code between defaults.

What an interface still *cannot* do: hold mutable instance state. So the decision rule is: need shared state or a constructor → abstract class; need a mix-in capability across unrelated hierarchies → interface. When both fit, prefer the interface for flexibility.

```
interface Greeter {  
    String name();  
    default String greet() { return "Hi, " + name(); } // Java 8+  
    static Greeter of(String n) { return () -> n; }  
}
```

GOTCHA

If a class inherits two default methods with the same signature from two interfaces, it must override to resolve the conflict — the "diamond problem." You can call a specific one with `Greeter.super.greet()`.

3 Access Modifiers

CORE

ENCAPSULATION

Q Explain the four access levels in Java and when you would use each.

A Java has four access levels controlling visibility, from most to least restrictive:

`private`, package-private (default, no keyword), `protected`, and `public`. They enforce encapsulation by limiting who can see a member.

private — visible only within the declaring class. This is your default for fields; it protects invariants and hides implementation. **default / package-private** — visible to any class in the same package. Useful for classes that collaborate closely within a module but should not be part of the public API. **protected** — visible within the package *and* to subclasses in other packages. It is meant for extension points a subclass needs. **public** — visible everywhere; this is your committed API surface, hardest to change later.

The guiding principle is "least privilege": start as restrictive as possible and widen only when a real need appears. A widely used public method is effectively frozen — you cannot narrow its access without breaking callers.

```
public class Account {
    private double balance;           // only Account
    protected void onOverdraft() {}   // subclasses may override
    void internalAudit() {}           // package-private helper
    public double getBalance() { return balance; }
}
```

GOTCHA

Top-level classes can only be `public` or package-private — never `private` or `protected`. Also, `protected` is more permissive than default because it adds cross-package subclass access.

4 static vs Instance Members

CORE

MEMORY

Q What is the difference between static and instance members, and what are all the uses of the static keyword?

A An **instance member** belongs to a specific object — each instance gets its own copy of a field, and instance methods can access `this`. A **static member** belongs to the class itself: there is exactly one copy shared across all instances, and it exists even if you never create an object.

The `static` keyword has several uses: **static fields** (shared class state / constants), **static methods** (utility functions that need no instance, like `Math.max`), **static nested classes** (nested classes that do not hold a reference to the outer instance), and **static initializer blocks** (run once when the class is loaded, to initialise complex static state). Java also has `import static` for importing static members by name.

Static members are resolved at compile time by the reference's declared type, so they are *not* polymorphic — you cannot override a static method, you can only hide it. A static method cannot use `this` or access instance members directly because there is no instance in scope.

```
class Counter {
    private static int count;           // shared across all instances
    static { count = 0; }               // static initializer, runs at class
load
    Counter() { count++; }
    static int total() { return count; } // no instance needed
}
```

GOTCHA

Mutable static fields are shared global state and a common source of thread-safety bugs. Also, calling a static method through an instance reference (`obj.staticMethod()`) compiles but is misleading — it is resolved by the declared type, not the object.

5 final vs finally vs finalize

CORE

KEYWORDS

Q What is the difference between final, finally, and finalize?

A They sound alike but are unrelated. **final** is a modifier, **finally** is a block in exception handling, and **finalize** is a (now deprecated) method on `Object`.

final means "cannot change." A `final` variable can be assigned once (a constant or effectively-immutable reference); a `final` method cannot be overridden; a `final` class cannot be subclassed (like `String`). Note that a final reference to an object still allows mutating the object's internal state — only the reference is frozen.

finally is a block attached to `try` that always executes, whether the try completes normally, throws, or even returns — used for cleanup like closing resources. The rare exception is if the JVM exits (`System.exit`) or the thread is killed.

finalize() was a callback the garbage collector *might* invoke before reclaiming an object. It is unreliable (no guarantee it runs, or when), hurts performance, and is deprecated since Java 9 and slated for removal. Use `try-with-resources` or `Cleaner` instead for cleanup.

```
final int MAX = 100;           // constant
try {
    risky();
} finally {
    cleanup();                 // always runs
}
```

GOTCHA

A `return` inside a `finally` block silently swallows exceptions and overrides any earlier return value — a classic trap. Never `return` or `throw` from `finally`.

6 Overloading vs Overriding

OOP

POLYMORPHISM

Q What is the difference between method overloading and overriding, including the rules for each?

A **Overloading** is compile-time (static) polymorphism: multiple methods with the same name but different parameter lists in the same class. **Overriding** is runtime (dynamic) polymorphism: a subclass provides a new implementation for an inherited method with the same signature.

Overloading rules: methods must differ in the number or types of parameters. Return type and access modifiers alone do *not* distinguish overloads — you cannot overload by return type only. Which overload runs is chosen by the compiler based on the *static* (declared) types of the arguments.

Overriding rules: same name, same parameter types, and a return type that is the same or a covariant subtype. The overriding method cannot reduce visibility (can widen it), and cannot throw broader checked exceptions than the overridden method. Which method runs is decided at runtime by the object's actual type (virtual dispatch). Always use `@Override` so the compiler catches signature mistakes.

```
class Base { Number id() { return 1; } }
class Sub extends Base {
    @Override Integer id() { return 2; } // covariant return, override
    Integer id(int seed) { return seed; } // overload, different params
}
```

GOTCHA

Overload resolution uses the declared type, so `print(null)` can be ambiguous or pick a surprising overload. And you cannot override a `static`, `final`, or `private` method — a same-named static method in a subclass merely *hides* it.

7 Why String Is Immutable

CORE

STRINGS

Q Why is String immutable in Java, and what is the string pool?

A `String` is immutable: once created, its contents never change. Any "modifying" operation returns a new `String`. This is enforced by the class being `final` with a private, internally-managed character array.

Immutability buys several things. **Security**: strings are used for file paths, class names, and network hosts; if they could mutate after a security check, an attacker could change them (TOCTOU). **Thread safety**: immutable objects can be shared freely across threads with no synchronization. **Caching of the hash code**: since content never changes, `String` caches its `hashCode()`, making it a fast, reliable `HashMap` key. **Interning**: the JVM can safely share duplicate literals.

The **string pool** (string intern pool) is a special area where the JVM stores unique string literals. When you write a literal like `"hello"`, the JVM checks the pool; if an equal string exists, it reuses that reference instead of allocating a new object. This saves memory. `new String("hello")` bypasses the pool and always allocates a fresh object on the heap; you can add it to the pool manually with `.intern()`.

```
String a = "hello";           // pooled
String b = "hello";           // same reference from pool
String c = new String("hello"); // new heap object
System.out.println(a == b);   // true
System.out.println(a == c);   // false
```

GOTCHA

Building strings in a loop with `+` creates many throwaway objects (each concat allocates). Use `StringBuilder`. Also, compile-time constant expressions like `"he" + "llo"` are folded and pooled, but concatenation of runtime variables is not.

8 String vs StringBuilder vs StringBuffer

CORE

STRINGS

Q When would you use String, StringBuilder, or StringBuffer?

A The distinction is **immutability** and **thread safety**. `String` is immutable.

`StringBuilder` and `StringBuffer` are mutable, growable character sequences — the difference between them is that `StringBuffer`'s methods are **synchronized** (thread-safe) while `StringBuilder`'s are not.

Use `String` for fixed text and as keys/values you pass around — its immutability makes it safe and cacheable. Use `StringBuilder` whenever you build a string through many appends, especially in loops, because it mutates one internal buffer instead of allocating a new `String` on every step. Repeated `+` concatenation in a loop is **$O(n^2)$** in the worst case; `StringBuilder` makes it **$O(n)$** .

Use `StringBuffer` only when multiple threads mutate the same buffer concurrently — a rare situation. Its synchronization adds overhead you almost never need; `StringBuilder` is the modern default. In practice, most modern code confines a builder to one method (one thread), so `StringBuilder` wins.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; i++) {
    sb.append(i).append(","); // mutates one buffer, O(n) total
}
String result = sb.toString();
```

GOTCHA

Modern `javac` compiles simple `a + b` to a `StringBuilder` (or `invokedynamic` since Java 9), so a single concatenation is fine — the problem is concatenation *inside loops*, where a new builder is created each iteration.

9 == vs .equals()

CORE

EQUALITY

Q What is the difference between == and .equals()?

A == compares **references** for object types — it asks "are these two variables pointing to the same object in memory?" For primitives, == compares values directly. .equals() is a *method* that compares **logical equality** as defined by the class.

The default `Object.equals()` just does == (identity). Classes that represent values — `String`, `Integer`, `LocalDate` — override `equals()` to compare contents. So `new String("a") == new String("a")` is `false` (two objects) but `.equals()` is `true` (same characters).

The practical rule: use `.equals()` to compare object contents and reserve == for primitives and for deliberate identity checks (e.g. comparing to `null`, or checking enum constants where identity equals value). Guard against NPE by calling `equals` on a known-non-null side or using `Objects.equals(a, b)`, which handles nulls.

```
String x = new String("a"), y = new String("a");
x == y           // false – different objects
x.equals(y)      // true – same content
Objects.equals(x, y) // true, null-safe
```

GOTCHA

Comparing boxed `Integer` with == is a classic bug: it works for small cached values (-128..127) but silently fails for larger ones. Always use `.equals()` or unbox to `int`.

10 equals() and hashCode() Contract

CORE

COLLECTIONS

Q What is the contract between `equals()` and `hashCode()`, and why must you override both together?

A The contract has one central rule: if two objects are equal by `equals()`, they must have the same `hashCode()`. The reverse need not hold — unequal objects *may* share a hash code (a collision), that is allowed and expected.

`equals()` itself must be reflexive, symmetric, transitive, consistent, and return `false` for `null`. `hashCode()` must be consistent (same object, same value while its state is unchanged) and must agree with `equals()`.

Why override both? Hash-based collections (`HashMap` , `HashSet`) first use `hashCode()` to find the bucket, then `equals()` to find the exact entry within it. If you override `equals()` but not `hashCode()`, two "equal" objects can land in different buckets, so a `HashSet` would store duplicates and `map.get(key)` would miss an entry that is logically present — a subtle, data-losing bug.

```
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Point p)) return false;
    return x == p.x && y == p.y;
}
@Override public int hashCode() { return Objects.hash(x, y); }
```

GOTCHA

Never use a mutable field in `equals` / `hashCode` if the object may change while stored in a hash set — its bucket becomes stale and it "disappears" from the collection. Keys should be immutable.

11 Autoboxing and the Integer Cache

CORE

PITFALLS

Q What are autoboxing and unboxing, and what is the Integer cache pitfall?

A **Autoboxing** is the compiler automatically converting a primitive to its wrapper (`int` → `Integer`); **unboxing** is the reverse. It lets you put primitives into generic collections like `List<Integer>` and mix the two seamlessly.

The pitfall: to save memory, `Integer.valueOf()` (which autoboxing calls) caches boxed values from **-128 to 127**. Two autoboxed `Integer`s in that range are the *same* cached object, so `==` returns `true`; outside that range, each box is a new object and `==` returns `false`. This makes `==` on `Integer` deceptively "work" in tests with small numbers and break in production.

A second hazard is the **NullPointerException** on unboxing: if an `Integer` is `null` and you use it in arithmetic or assign it to an `int`, the JVM tries to unbox `null` and throws. A third is performance — boxing in a hot loop allocates objects and adds overhead, so prefer primitives in tight numeric code.

```
Integer a = 100, b = 100;
System.out.println(a == b); // true - cached (-128..127)
Integer c = 200, d = 200;
System.out.println(c == d); // false - different objects
System.out.println(c.equals(d)); // true - always use equals
```

GOTCHA

`Integer x = null; int y = x;` throws `NullPointerException` at the unboxing point — often surprising because there is no explicit method call. Watch this with map lookups that may return `null`.

12 Primitives vs Wrapper Types

CORE

MEMORY

Q What is the difference between primitive types and wrapper types, and when should you use each?

A Java has eight **primitives** (`byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`) that store raw values directly, and each has a corresponding **wrapper** class (`Integer`, `Long`, `Double`, `Boolean` , etc.) that is a real object on the heap.

Primitives are faster and use less memory — an `int` is 4 bytes of stack/inline storage, while an `Integer` is a heap object with header overhead plus a reference. Primitives cannot be `null` and have sensible defaults (`0` , `false`). Wrappers can be `null` , which is exactly why generics and collections require them: `List<int>` is illegal, you must use `List<Integer>` , because generic type parameters must be reference types.

Rule of thumb: use primitives by default for local variables, fields, loop counters, and performance-sensitive code. Use wrappers when you need nullability (e.g. an optional database column), when storing in collections, or when you need the wrapper's utility methods (`Integer.parseInt` , `Long.MAX_VALUE`). Wrappers also enable the null-vs-zero distinction, which primitives cannot express.

```
int count = 0;                // cannot be null, fast
Integer nullableAge = null;    // "unknown" vs 0
List<Integer> ids = new ArrayList<>(); // collections need wrappers
```

GOTCHA

A `Map<String,Integer>.get(missingKey)` returns `null` ; assigning it to an `int` throws NPE via unboxing. Also beware `float` / `double` for money — use `BigDecimal` to avoid rounding error.

13 Checked vs Unchecked Exceptions

CORE

EXCEPTIONS

Q What is the difference between checked and unchecked exceptions?

A Java exceptions form a hierarchy under `Throwable`. **Checked exceptions** extend `Exception` (but not `RuntimeException`) and must be either caught or declared in the method's `throws` clause — the compiler enforces this. **Unchecked exceptions** extend `RuntimeException` (or `Error`) and require no declaration.

The intent: checked exceptions represent *recoverable* conditions the caller should reasonably anticipate and handle — `IOException`, `SQLException` — like a missing file or a network hiccup. Unchecked exceptions represent *programming errors* — `NullPointerException`, `IllegalArgumentException`, `IndexOutOfBoundsException` — bugs the caller should fix in code, not catch.

`Error` (e.g. `OutOfMemoryError`, `StackOverflowError`) is unchecked too and signals serious JVM problems you normally should not catch. Design guidance: do not overuse checked exceptions; they clutter APIs and often get swallowed. Many modern frameworks favour unchecked exceptions for cleaner signatures, wrapping lower-level checked exceptions.

```
// checked: compiler forces handling
void read() throws IOException { Files.readString(path); }

// unchecked: a bug, no declaration needed
int x = arr[10]; // ArrayIndexOutOfBoundsException
```

GOTCHA

Catching `Exception` (or worse, `Throwable`) too broadly hides bugs and can swallow `InterruptedException`. Never write an empty catch block — at minimum log or rethrow wrapped.

14 try-with-resources and AutoCloseable

CORE

EXCEPTIONS

Q What is try-with-resources, and how does it relate to AutoCloseable?

A **try-with-resources** (Java 7+) is a form of `try` that declares one or more resources in parentheses and automatically closes them when the block exits — normally or via exception. It replaces the error-prone manual `try/finally` close pattern.

A resource qualifies if it implements `AutoCloseable` (or its subtype `Closeable`), which declares a single `close()` method. The compiler generates the cleanup: resources are closed in **reverse** order of declaration, and closing happens before any catch/finally you attach. This guarantees no leaked file handles, sockets, or connections even if the body throws.

A key correctness feature is **suppressed exceptions**: if the body throws and `close()` also throws, the body's exception is propagated as primary and the close exception is attached via `getSuppressed()` — so you never lose the original error. The old manual pattern could mask the real exception with a close failure.

```
try (var in = Files.newInputStream(path);
    var out = Files.newOutputStream(dst)) {
    in.transferTo(out);
} // out.close() then in.close(), automatically
```

GOTCHA

Since Java 9 you can use an *effectively final* variable declared outside the try: `try (conn) { ... }`. But a resource assigned to `null` in the header is simply skipped at close — no NPE.

15 Collections Framework Overview

COLLECTIONS

CORE

Q Give an overview of the Java Collections Framework — List, Set, Map, and Queue.

A The Collections Framework is a set of interfaces and implementations for storing groups of objects. The root interface is `Collection`, with `Map` as a parallel hierarchy (a map is not a `Collection` because it stores key-value pairs, not single elements).

List — an ordered sequence allowing duplicates and indexed access (`ArrayList`, `LinkedList`). **Set** — a collection with no duplicates (`HashSet` unordered, `LinkedHashSet` insertion-ordered, `TreeSet` sorted). **Queue / Deque** — elements processed in an order, typically FIFO (`ArrayDeque`, `LinkedList`, `PriorityQueue` for priority order). **Map** — key → value associations with unique keys (`HashMap`, `LinkedHashMap`, `TreeMap`).

Choosing is about access patterns and ordering: need fast random access → `ArrayList`; need uniqueness → a `Set`; need key lookup → a `Map`; need FIFO/LIFO → a `Deque`; need sorted iteration → the `Tree` variants. There are also `java.util.concurrent` versions (`ConcurrentHashMap`, `CopyOnWriteArrayList`) for thread-safe use.

```
List<String> list = new ArrayList<>(); // ordered, duplicates
Set<String> set = new HashSet<>(); // unique, unordered
Map<String,Integer> map = new HashMap<>(); // key -> value
Deque<String> stack = new ArrayDeque<>(); // LIFO or FIFO
```

GOTCHA

The legacy `Vector`, `Hashtable`, and `Stack` are synchronized and largely obsolete — prefer `ArrayList` / `HashMap` / `ArrayDeque`, adding explicit synchronization or concurrent collections only when needed.

16 ArrayList vs LinkedList

COLLECTIONS

PERFORMANCE

Q What is the difference between ArrayList and LinkedList, and when would you choose each?

A Both implement `List`, but they have very different internal structures. `ArrayList` is backed by a **resizable array**; `LinkedList` is a **doubly-linked list** of nodes, each holding a value and pointers to its neighbours.

`ArrayList` gives **$O(1)$** random access by index because it computes a memory offset directly. Adding at the end is amortised **$O(1)$** , but inserting or removing in the middle is **$O(n)$** due to shifting elements. `LinkedList` has **$O(1)$** insertion/removal *once you have the node* (e.g. at the ends), but random access by index is **$O(n)$** because it must walk the chain from the nearest end.

In practice `ArrayList` wins almost always: arrays have excellent cache locality and no per-element node overhead, whereas `LinkedList` allocates a node object per element and its pointers scatter across the heap, hurting real-world performance. Choose `LinkedList` mainly when you need a `Deque` /queue with heavy add/remove at both ends — though `ArrayDeque` usually beats it there too.

```
List<Integer> a = new ArrayList<>();
a.get(i);           //  $O(1)$  random access
a.add(x);           // amortized  $O(1)$ 
a.add(0, x);        //  $O(n)$  – shifts everything
```

GOTCHA

People assume `LinkedList` is faster for "lots of inserts," but `list.add(i, x)` is still **$O(n)$** because finding index `i` requires traversal. The **$O(1)$** claim only holds when you already hold an iterator/node at that position.

17 How HashMap Works Internally

COLLECTIONS

INTERNALS

Q How does a HashMap work internally — buckets, hashing, collisions, and treeification?

A A `HashMap` stores entries in an array of **buckets**. To place a key, it calls the key's `hashCode()`, applies an internal spreading function to mix the high bits, then maps that to a bucket index with `hash & (capacity - 1)` (capacity is a power of two, so this is a fast bitmask).

When two keys land in the same bucket (a **collision**), they are chained. Historically this was a linked list; since Java 8, if a single bucket grows past a threshold (8 entries) and the table is large enough (≥ 64), that bucket is **treeified** into a balanced red-black tree, improving worst-case lookup from $O(n)$ to $O(\log n)$. Lookups compare hashes first, then use `equals()` to find the exact key. Average get/put is $O(1)$.

As entries grow, once `size > capacity × loadFactor` (default load factor 0.75), the map **resizes**: it doubles capacity and rehashes all entries into the larger table. Sizing the map up front (`new HashMap<>(expectedSize)`) avoids repeated costly resizes.

```
// simplified index computation
int h = key.hashCode();
h = h ^ (h >>> 16);           // spread high bits
int index = h & (capacity - 1); // bucket
```

GOTCHA

A bad or constant `hashCode()` collapses everything into one bucket, degrading to $O(n)$ (or $O(\log n)$ after treeify). And `HashMap` is not thread-safe — concurrent writes can corrupt it; use `ConcurrentHashMap`.

18 HashSet vs TreeSet vs LinkedHashSet

COLLECTIONS

SETS

Q What are the differences between HashSet, TreeSet, and LinkedHashSet?

A All three implement `Set` (no duplicates), but differ in ordering guarantees and performance, because they are backed by different structures.

HashSet is backed by a `HashMap`. It gives **$O(1)$** average add/contains/remove but **no ordering** — iteration order is unspecified and can change on resize. **LinkedHashSet** extends it with a linked list threading the entries, so it preserves **insertion order** while keeping **$O(1)$** operations, at a small memory cost. **TreeSet** is backed by a red-black tree and keeps elements in **sorted order** (natural ordering or a supplied `Comparator`), with **$O(\log n)$** operations and useful navigation methods like `first()`, `ceiling()`, `headSet()`.

Choose `HashSet` for plain fast membership when order does not matter; `LinkedHashSet` when you want predictable iteration matching insertion (e.g. deduplicating while preserving order); `TreeSet` when you need sorted iteration or range queries.

```
Set<String> h = new HashSet<>();           // no order, O(1)
Set<String> l = new LinkedHashSet<>();     // insertion order, O(1)
Set<String> t = new TreeSet<>();           // sorted, O(log n)
```

GOTCHA

`TreeSet` uses `compareTo` / `Comparator` — **not equals** — to decide equality. If the comparator says two distinct objects "compare equal" (return 0), the set treats them as duplicates, which can silently drop elements.

19 Comparable vs Comparator

CORE

SORTING

Q What is the difference between Comparable and Comparator?

A Both define ordering, but from different places. `Comparable` is implemented by the class *being* sorted and defines its single **natural ordering** via `compareTo(other)`.

`Comparator` is a separate object defining an **external, alternative ordering** via `compare(a, b)`.

Use `Comparable` when a type has one obvious default order — `Integer` by value, `String` lexicographically. Use `Comparator` when you need multiple or context-specific orderings (sort people by name, then by age), when you cannot modify the class, or when the order is a caller's concern rather than the type's identity. Since Java 8, comparators compose fluently with `comparing`, `thenComparing`, and `reversed`.

Both return a negative, zero, or positive int meaning "less / equal / greater." A method like `Collections.sort(list)` uses natural order (requires `Comparable`); `Collections.sort(list, cmp)` uses the supplied `Comparator`.

```
class Person implements Comparable<Person> {
    int age; String name;
    @Override public int compareTo(Person o) { return Integer.compare(age,
o.age); }
}
// external, composed ordering
list.sort(Comparator.comparing((Person p) -> p.name).thenComparingInt(p ->
p.age));
```

GOTCHA

Implementing `compareTo` as `a - b` can **overflow** for large ints and give wrong signs — use `Integer.compare(a, b)`. Also keep ordering consistent with `equals`, or sorted collections behave oddly.

20 Generics and Wildcards (PECS)

CORE

GENERICS

Q Explain generics and wildcards — what do `? extends` and `? super` mean, and what is PECS?

A **Generics** give compile-time type safety to code that works over many types, letting you write `List<String>` instead of a raw `List` and catch type errors at compile time rather than as runtime `ClassCastException`. Java implements them with **type erasure**: type parameters exist at compile time and are erased to their bounds (usually `Object`) at runtime.

Generics are **invariant**: `List<Integer>` is *not* a subtype of `List<Number>`, even though `Integer` is a `Number`. Wildcards restore flexibility. `? extends T` is an **upper-bounded** wildcard — an unknown type that is *some subtype* of T; you can safely **read** items as T, but cannot add (the compiler does not know the exact type). `? super T` is **lower-bounded** — some supertype of T; you can safely **write** T into it, but reads come back only as `Object`.

The mnemonic is **PECS** — "**P**roducer **E**xtends, **C**onsumer **S**uper." If a parameter *produces* values you read out, use `extends`; if it *consumes* values you put in, use `super`. This is exactly how `Collections.copy(dest, src)` is declared. Use an unbounded `<?>` when you only need to work with the structure, not the element type.

```
// Producer Extends, Consumer Super
static <T> void copy(List<? super T> dest, List<? extends T> src) {
    for (T t : src) dest.add(t); // read from src, write to dest
}
```

GOTCHA

Because of erasure you cannot do `new T[]`, `instanceof List<String>`, or catch a generic exception type. And you cannot add anything except `null` to a `List<? extends Number>` — a frequent point of confusion.

21 Processes, Threads & Thread Creation

CONCURRENCY

Q What is the difference between a process and a thread, and what are the ways to create a thread in Java?

A A **process** is an independent program in execution with its own isolated memory space (heap, code, data). A **thread** is a lightweight unit of execution that lives inside a process and shares that process's heap and static data with sibling threads, while keeping its own stack, program counter, and local variables. The reason threads are cheap and processes are expensive comes down to this sharing: creating a thread does not duplicate the address space, and switching between threads of the same process is far cheaper than a full process context switch.

Because threads share the heap, communication between them is trivial (just read/write shared objects) but dangerous — this shared mutable state is the root of every race condition, and it is precisely why we need synchronization. Processes, by contrast, are isolated by the OS and must communicate through slower IPC mechanisms (pipes, sockets, shared memory).

Java gives you three main ways to define work for a thread. You can **extend Thread** and override `run()`, but this burns your single inheritance slot and couples the task to the mechanism. Better is to **implement Runnable** — a functional interface whose `run()` returns nothing and throws no checked exceptions — and hand it to a thread or executor. When you need a **result or checked exception**, implement `Callable<V>`, whose `call()` returns a value and is submitted to an `ExecutorService` to yield a `Future<V>`.

```
Runnable r = () -> System.out.println("work");
new Thread(r).start();           // Runnable: no result

Callable<Integer> c = () -> 42;  // Callable: returns a value
Future<Integer> f = pool.submit(c);
int result = f.get();           // blocks for the answer
```

GOTCHA

Call `start()` , never `run()` . Calling `run()` directly executes the code on the *current* thread — no new thread is ever created. Also, a `Thread` object can only be started once; calling `start()` twice throws `IllegalThreadStateException` .

22 Thread Lifecycle & States

CONCURRENCY

Q What are the states in a thread's lifecycle?

A A Java thread is always in exactly one of six states defined by the `Thread.State` enum, and understanding them matters because the state tells you **why a thread is not making progress** — which is the first question in any concurrency bug. The states are NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED.

A thread is **NEW** after construction but before `start()`. Once started it becomes **RUNNABLE** — note this single state covers both "running on a CPU" and "ready and waiting for a CPU slot"; Java deliberately does not distinguish them because scheduling is the OS's job. It is also worth knowing that a thread blocked on I/O appears as RUNNABLE to the JVM, since the JVM cannot see the OS-level wait.

The three non-runnable live states differ by cause. **BLOCKED** means the thread is waiting to acquire an intrinsic monitor lock to enter a `synchronized` region. **WAITING** means it called `wait()`, `join()`, or `park()` with no timeout and will stay there until explicitly signaled. **TIMED_WAITING** is the same but with a deadline, entered via `sleep(ms)`, `wait(ms)`, or `join(ms)`. Finally **TERMINATED** means `run()` has completed or thrown.

```
NEW --start()--> RUNNABLE --run() ends--> TERMINATED
RUNNABLE <--> BLOCKED           // contending for a lock
RUNNABLE <--> WAITING           // wait()/join()/park()
RUNNABLE <--> TIMED_WAITING      // sleep(t)/wait(t)
```

GOTCHA

A common trap: `Thread.sleep()` does **not release any locks** the thread holds — it only pauses the thread. Only `wait()` releases the monitor. Sleeping inside a `synchronized` block is a reliable way to starve every other thread waiting on that lock.

23 The synchronized Keyword

CONCURRENCY

Q How does synchronized work, and what is the difference between synchronized methods and blocks?

A `synchronized` provides mutual exclusion using an **intrinsic lock** (also called a monitor) that every Java object carries. Only one thread can hold a given object's monitor at a time; others attempting to enter a region guarded by that same monitor go BLOCKED until it is released. The keyword solves two problems at once: it guarantees **atomicity** of the guarded region and it establishes **happens-before** visibility — everything a thread does before releasing a monitor is visible to the next thread that acquires it.

A **synchronized instance method** implicitly locks on `this`; a **synchronized static method** locks on the `Class` object. A **synchronized block** lets you name the lock object explicitly and narrow the critical section to only the lines that need it, which reduces contention and can improve throughput. Prefer blocks over whole methods when only part of the method touches shared state.

Intrinsic locks are **reentrant**: a thread already holding a monitor can re-acquire it (e.g., one synchronized method calling another on the same object) without deadlocking itself. The lock counts nesting depth and is fully released only when the count returns to zero.

```
public void transfer() {
    doSetup();                // no lock needed here
    synchronized (balanceLock) { // lock only the critical part
        balance += amount;
    }
}
```

GOTCHA

Never synchronize on a mutable field or on a pooled/interned object like a boxed `Integer` or a `String` literal. If two unrelated pieces of code lock on the same interned `"lock"` string, they will contend accidentally. Use a `private final Object lock = new Object();` dedicated to the purpose.

24 The volatile Keyword

CONCURRENCY

Q What does volatile guarantee, and what does it not?

A `volatile` solves the **visibility** problem. Without it, each thread may cache a field in a register or CPU cache, so one thread's write may never become visible to another, causing bugs like an infinite loop reading a stale `stop` flag. Marking a field volatile forces every read to come from and every write to go to **main memory**, guaranteeing that a write by one thread is immediately visible to all others.

It also provides an **ordering** guarantee: a volatile write happens-before every subsequent volatile read of the same field, and the JVM inserts memory barriers that prevent surrounding reads/writes from being reordered across it. This is what makes volatile usable for flags and for the reference in the double-checked-locking singleton idiom.

What volatile does **not** give you is atomicity of compound actions. `count++` is a read-modify-write; declaring `count` volatile makes each read and each write visible but does nothing to stop two threads from reading the same value and both writing back the same increment. For that you need `synchronized` or an `AtomicInteger`.

```
private volatile boolean running = true;    // visibility: OK

public void stop() { running = false; }      // seen by worker at once
public void loop() { while (running) { /* ... */ } }

private volatile int n;
n++;    // STILL a race – not atomic despite volatile
```

GOTCHA

Rule of thumb: `volatile` is correct only when the value written does not depend on its current value (a flag, a published reference), or when a single thread writes and others only read. The moment multiple writers derive the new value from the old, reach for atomics or locks.

25 Java Memory Model & Happens-Before

CONCURRENCY

Q What is the Java Memory Model, and what does "happens-before" mean?

A The **Java Memory Model (JMM)** is the specification that defines when a write performed by one thread is guaranteed to be visible to a read performed by another. It exists because CPUs and compilers aggressively reorder and cache memory operations for speed; without a formal contract, "correct" multithreaded code would depend on hardware. The JMM draws the line between legal optimizations and guaranteed visibility.

The central concept is **happens-before**, a partial ordering between actions. If action A happens-before action B, then A's memory effects are visible to and ordered before B. If two accesses to the same data are *not* ordered by happens-before and at least one is a write, you have a **data race** and the result is undefined.

The key happens-before rules you must know: **program order** within a single thread; a monitor **unlock happens-before every subsequent lock** of the same monitor; a **volatile write happens-before every subsequent read** of that field; `Thread.start()` happens-before everything the started thread does; and everything a thread does happens-before another thread returning from its `join()`. Happens-before is also transitive.

```
// Thread A
data = 42;           // (1) plain write
ready = true;        // (2) VOLATILE write

// Thread B
if (ready)           // (3) volatile read sees true
    use(data);        // (1) happens-before (3): sees 42
```


GOTCHA

Absence of a crash is not proof of correctness. A racy program can appear to work for years and then fail after a JIT recompilation or on a different CPU. Reason about correctness through happens-before edges, not through observed behavior.

26 Deadlock

CONCURRENCY

Q What causes a deadlock and how do you prevent it?

A A **deadlock** is a state where two or more threads are each blocked forever, waiting for a lock that another one of them holds. The classic case: thread 1 holds lock A and wants B, while thread 2 holds B and wants A. Neither can proceed and neither will release, so both wait indefinitely. Understanding it requires the four **Coffman conditions**, all of which must hold simultaneously for deadlock to be possible.

Those conditions are **mutual exclusion** (a resource is held exclusively), **hold-and-wait** (a thread holds one lock while requesting another), **no preemption** (locks can't be forcibly taken away), and **circular wait** (a cycle of threads each waiting on the next). Break any one of them and deadlock becomes impossible.

The most practical prevention breaks the circular wait by imposing a **global lock ordering**: every thread that needs multiple locks always acquires them in the same agreed order (e.g., by object identity hash or account ID). Other techniques: use `tryLock(timeout)` from `ReentrantLock` so a thread can back off and retry instead of blocking forever, and minimize lock scope so you rarely hold two locks at once.

```
// Enforce a consistent order to break circular wait
Account first = a.id < b.id ? a : b;
Account second = a.id < b.id ? b : a;
synchronized (first) {
    synchronized (second) { transfer(a, b); }
}
```

GOTCHA

Do not confuse deadlock with **livelock** (threads keep responding to each other and changing state but make no progress) or **starvation** (a thread never gets CPU or a lock because others monopolize it). A thread dump (`jstack`) will explicitly report "Found one Java-level deadlock" — use it.

27 wait(), notify() & notifyAll()

CONCURRENCY

Q How do wait(), notify(), and notifyAll() work?

A These `Object` methods implement the low-level **condition-wait** protocol that lets threads coordinate on a shared object's monitor. `wait()` makes the current thread release the monitor and go WAITING until another thread calls `notify()` or `notifyAll()` on the same object. The reason they belong on `Object` and not `Thread` is that they operate on the object's intrinsic monitor — which is why you may only call them while **holding that monitor**, i.e., inside a synchronized block on that object, else you get `IllegalMonitorStateException`.

The single most important rule: **always wait in a loop that re-checks the condition**, never in an `if`. This is because of *spurious wakeups* (a thread can wake without notification) and because by the time a notified thread reacquires the lock, another thread may have already invalidated the condition. The loop guarantees you only proceed when the predicate genuinely holds.

`notify()` wakes one arbitrary waiting thread; `notifyAll()` wakes all of them. Prefer `notifyAll()` unless you can prove all waiters are interchangeable, because `notify()` can wake the "wrong" thread and leave a thread that could have proceeded asleep — a subtle lost-wakeup bug.

```
synchronized (queue) {
    while (queue.isEmpty()) // loop, not if!
        queue.wait();      // releases lock, sleeps
    process(queue.remove());
}
// producer side:
synchronized (queue) { queue.add(item); queue.notifyAll(); }
```

GOTCHA

In modern code prefer higher-level tools — `BlockingQueue` , `CountDownLatch` , or a `Lock` with `Condition` objects (`await()` / `signal()`). They are harder to misuse and let you have multiple distinct wait sets on one lock.

28 ExecutorService & Thread Pools

CONCURRENCY

Q Why use an `ExecutorService` and thread pools instead of creating threads directly?

A Creating a raw `new Thread()` per task is expensive and unbounded. Each thread costs memory (a stack, often ~1 MB by default) and OS resources, and under load you can spawn thousands, exhaust memory, and thrash the scheduler. The **Executor framework** solves this by **decoupling task submission from execution**: you submit `Runnable` / `Callable` tasks and a pool of reused worker threads runs them.

A **thread pool** bounds the number of live threads and reuses them across many tasks, amortizing creation cost and preventing resource exhaustion. Tasks queue up when all workers are busy, giving you natural backpressure. You get lifecycle management (`shutdown()` , `awaitTermination()`), `Future` results, and scheduled execution for free.

Prefer configuring a `ThreadPoolExecutor` directly (core size, max size, queue, rejection policy) over the convenience factories. In particular, `Executors.newFixedThreadPool` and `newSingleThreadExecutor` use an **unbounded `LinkedBlockingQueue`** , so a flood of tasks can queue until you run out of memory — a bounded queue with a sensible rejection policy is safer in production.

```
ExecutorService pool = new ThreadPoolExecutor(
    4, 8,                                // core, max threads
    60L, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(100), // bounded queue
    new ThreadPoolExecutor.CallerRunsPolicy());
pool.submit(task);
pool.shutdown();                        // stop accepting, finish queued
```

GOTCHA

Always shut the pool down, or the non-daemon worker threads keep the JVM alive forever. And size CPU-bound pools near the core count, but I/O-bound pools much larger since threads spend most time blocked. On Java 21+, consider

`newVirtualThreadPerTaskExecutor()` for massively concurrent blocking I/O.

29 Future vs CompletableFuture

CONCURRENCY

Q What is the difference between Future and CompletableFuture?

A A `Future<T>` is a handle to a result that will be available later. Its weakness is that it is **passive and blocking**: the only way to get the value is `get()`, which blocks the calling thread until the task finishes. You cannot register a callback, chain another step, or combine several futures without ugly polling with `isDone()`. This makes it awkward for building asynchronous pipelines.

`CompletableFuture<T>` (Java 8+) implements both `Future` and `CompletionStage`, adding a rich, **non-blocking** fluent API. You attach continuations that fire *when the result is ready* — `thenApply` to transform, `thenCompose` to chain a dependent async call (flat-map), `thenCombine` to merge two independent results, and `exceptionally` / `handle` for error recovery. This lets you express a whole dataflow declaratively without ever blocking a thread.

It is also **completable** — the name — meaning you can construct one and complete it manually from any code (e.g., from a callback-based library), bridging non-Future APIs into the pipeline. Continuations run on the common `ForkJoinPool` by default, or on an executor you supply via the `...Async` variants.

```
CompletableFuture
    .supplyAsync(() -> fetchUser(id), pool)
    .thenApply(User::name)                // transform
    .thenCompose(n -> lookupAsync(n))     // chain async
    .exceptionally(ex -> "unknown")      // recover
    .thenAccept(System.out::println);    // consume, no block
```

GOTCHA

Confusing `thenApply` with `thenCompose` : use `thenApply` when your function returns a plain value, and `thenCompose` when it returns another

`CompletableFuture` — otherwise you get a nested

`CompletableFuture<CompletableFuture<T>>` .

30 ConcurrentHashMap vs Synchronized HashMap

CONCURRENCY

Q How does ConcurrentHashMap differ from a synchronized HashMap, and how does it work?

A A `Collections.synchronizedMap(new HashMap<>())` wraps every method in a single lock on the whole map. This is correct but a **throughput bottleneck**: only one thread can touch the map at a time, even for reads, so concurrency collapses to serial access. `ConcurrentHashMap` (CHM) is engineered for high concurrency by **avoiding a single global lock**.

Modern CHM (Java 8+) locks at the granularity of an individual **hash bucket (bin)**, using the first node of the bin as the lock and a CAS to install the first node in an empty bin. Reads are **entirely lock-free** — nodes hold `volatile` values so readers see up-to-date data without synchronizing. The result is that many threads can read and write different buckets truly in parallel, giving far better scalability than the synchronized wrapper.

Two behavioral differences matter. CHM **forbids null keys and values** (a null return from `get()` would be ambiguous under concurrency — absent vs. present-but-null). And its iterators are **weakly consistent**, not fail-fast: they never throw `ConcurrentModificationException` and reflect some but not necessarily all concurrent updates. Compound operations should use its atomic methods.

```
ConcurrentHashMap<String,Integer> m = new ConcurrentHashMap<>();
m.putIfAbsent("k", 0);                               // atomic
m.compute("k", (k, v) -> v + 1);                       // atomic read-modify-write
m.merge("k", 1, Integer::sum);                         // atomic counter idiom
```

GOTCHA

Each CHM method is atomic, but a sequence of them is not. `if (!m.containsKey(k)) m.put(k, v);` is a race — two threads can both pass the check. Use `putIfAbsent`, `compute`, or `merge` to make the whole operation atomic.

31 Atomic Classes & CAS

CONCURRENCY

Q What are the atomic classes and how does compare-and-swap (CAS) work?

A The `java.util.concurrent.atomic` package (`AtomicInteger` , `AtomicLong` , `AtomicReference` , ...) provides variables whose read-modify-write operations are atomic **without locks**. The reason they can be lock-free is a hardware instruction called **compare-and-swap (CAS)**, exposed through low-level intrinsics. This gives you thread safety with much less overhead than a monitor under moderate contention.

CAS takes three arguments: a memory location, the **expected** current value, and the **new** value. Atomically, it checks whether the location still equals the expected value; if so it writes the new value and reports success, otherwise it does nothing and reports failure. Atomic classes wrap this in a **retry loop**: read the current value, compute the new one, attempt the CAS, and if it fails (someone else changed it) loop and try again with the fresh value. This is *optimistic* concurrency — assume no conflict, detect and retry if there was one.

The tradeoff is that under **very high contention** many threads keep failing and retrying, wasting CPU. For hot counters, `LongAdder` scales better: it spreads increments across multiple internal cells and sums them only when you read, trading exact instantaneous reads for far less contention.

```
private final AtomicInteger n = new AtomicInteger();
n.incrementAndGet();           // lock-free ++, via CAS loop

// what getAndIncrement does internally:
int cur;
do { cur = n.get(); }
while (!n.compareAndSet(cur, cur + 1)); // retry on conflict
```

GOTCHA

The **ABA problem**: a value changes from A to B and back to A between your read and CAS, so the CAS succeeds even though state churned underneath. When identity of intervening changes matters, use `AtomicStampedReference`, which pairs the reference with a version stamp.

32 ThreadLocal

CONCURRENCY

Q What is ThreadLocal, when is it useful, and what are its pitfalls?

A `ThreadLocal<T>` gives each thread its **own independent copy** of a variable. Instead of sharing one object across threads (and needing synchronization), every thread that calls `get()` sees a value private to itself. This sidesteps contention entirely for per-thread state and is the standard way to make an inherently non-thread-safe object usable concurrently — the classic example being `SimpleDateFormat`, which is not thread-safe but can be given one instance per thread.

Its common uses are carrying **implicit context** down a call stack without threading it through every method signature — a security principal, a transaction, a request/trace ID in web servers — and caching expensive-to-create, non-shareable helpers. Under the hood, each `Thread` holds a `ThreadLocalMap` keyed weakly by the `ThreadLocal` object.

The major pitfall is **memory leaks in thread pools**. Pool threads live for the life of the application and are reused, so a value you set on one task lingers into the next unrelated task — both leaking memory and, worse, **leaking data** across requests. You must **always** `remove()` in a `finally` block when done.

```
static final ThreadLocal<SimpleDateFormat> FMT =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));

try {
    ctx.set(requestId);
    handle();
} finally {
    ctx.remove();           // MUST clean up in pooled threads
}
```

GOTCHA

`InheritableThreadLocal` copies values to *child* threads at creation time, but that does not help pooled executors, where worker threads are created up front and reused — the child never re-inherits per task. This is a frequent source of "context is null" or "wrong tenant" bugs.

33 JVM Architecture

JVM

Q Describe the JVM architecture: class loader subsystem, execution engine, and JIT.

A The JVM is the abstract computing machine that runs Java bytecode, and it has three main subsystems: the **class loader subsystem**, the **runtime data areas** (memory), and the **execution engine**. The reason Java is "write once, run anywhere" is that `javac` compiles to platform-neutral bytecode and each platform's JVM translates that bytecode to native instructions at runtime.

The **class loader subsystem** performs loading, linking, and initialization. Loading finds the `.class` bytes; linking does verification (bytecode safety checks), preparation (allocating static fields with defaults), and resolution of symbolic references; initialization runs static initializers. Loaders follow the **parent-delegation model** — Bootstrap → Platform → Application — where each loader asks its parent first, which prevents core classes from being spoofed.

The **execution engine** runs the bytecode. It starts by **interpreting** bytecode instruction by instruction (fast startup), while a profiler watches for "hot" methods. The **JIT compiler** then compiles those hot paths to optimized native code (HotSpot uses tiered compilation with C1 and C2 compilers), so long-running code approaches native speed. A garbage collector reclaims unused heap objects.

```
Source.java --javac--> Source.class (bytecode)
      |
ClassLoader (load, link, init)
      |
Execution Engine: Interpreter --hot?--> JIT --> native code
      |
Garbage Collector reclaims heap
```

GOTCHA

JIT optimizations are **speculative**: HotSpot may inline a method or assume a branch is never taken based on profiling, then *deoptimize* back to the interpreter if the assumption breaks. This is why a Java program can appear to change performance characteristics as it "warms up," and why microbenchmarks need a warmup phase (use JMH).

34 JVM Memory Areas

JVM

Q What are the JVM's runtime memory areas?

A The JVM divides memory into areas with different lifetimes and sharing rules; knowing them tells you where an object lives and which errors it can cause. The two you cite constantly are the **heap** and the **stack**. The heap is a single region **shared by all threads** where every object and array is allocated and where garbage collection operates. When it fills, you get `OutOfMemoryError: Java heap space`.

Each thread gets its own **JVM stack**, made of stack frames — one per method call — holding local variables, operands, and the return address. Because it is per-thread, local variables are inherently thread-safe. Deep or infinite recursion overflows it, giving `StackOverflowError`. Each thread also has a **PC register** holding the address of the current bytecode instruction, so the JVM knows where to resume after a context switch.

Metaspace (which replaced PermGen in Java 8) holds class metadata — the loaded class structures, method bytecode, and the runtime constant pool. It lives in **native memory**, not the heap, and grows dynamically, so class-loader leaks show up as `OutOfMemoryError: Metaspace`. There is also a small per-thread native method stack for JNI calls.

Shared by all threads:

```
+-----+
|  Heap  |
| (objects) |
+-----+
| Metaspace |
| (class meta)|
+-----+
```

Per thread:

```
+-----+ +-----+
| JVM Stack | | PC |
| (frames) | +-----+
+-----+
(locals live on the stack;
objects live on the heap)
```


GOTCHA

A common interview trap: "where does a local `Object o = new Object();` live?" The **reference variable** `o` sits on the thread's stack, but the **object it points to lives on the shared heap**. Only primitives and references are stored directly in stack frames.

35 Garbage Collection

JVM

Q How does garbage collection work, and what are the major collectors?

A Garbage collection automatically reclaims heap memory occupied by objects that are no longer **reachable**. The core idea is **reachability**, not reference counting: starting from GC roots (stack locals, static fields, active threads), the collector traces every object reachable through references; anything not reached is garbage. This is why circular references between two dead objects are still collected — they are unreachable from any root.

Most collectors are **generational**, exploiting the *weak generational hypothesis*: most objects die young. The heap is split into a **young generation** (Eden + two survivor spaces) and an **old generation**. New objects go into Eden; frequent, cheap *minor GCs* copy the few survivors between survivor spaces and promote long-lived ones to old gen, which is collected by rarer, costlier *major/full GCs*. Collectors use mark-sweep-compact style algorithms and copying for the young gen.

The major collectors: **G1** (the default since Java 9) divides the heap into many equal regions and collects the ones with the most garbage first ("garbage first"), aiming to meet a soft pause-time target while still doing some stop-the-world work. **ZGC** and **Shenandoah** are low-latency collectors that do most marking and relocation *concurrently* with the application, keeping pauses under a few milliseconds even on very large heaps.

```
Young Gen: [ Eden | S0 | S1 ] --minor GC (frequent, fast)-->
Old Gen:   [   tenured   ] --major GC (rare, slower)
Roots: stack locals, statics, live threads --> trace reachable
Unreachable objects (incl. cycles) --> reclaimed
```

GOTCHA

GC frees *unreachable* objects, not *unused* ones. A forgotten entry in a long-lived static `Map`, or a listener never unregistered, stays reachable and leaks despite never being touched. Also, `System.gc()` is only a hint — never rely on it to force collection.

36 Lambdas & Functional Interfaces

JAVA 8+

Q What are lambda expressions and functional interfaces?

A A **functional interface** is an interface with exactly one abstract method (a SAM type), optionally annotated `@FunctionalInterface` to have the compiler enforce that. A **lambda expression** is concise syntax for providing an implementation of that single method — an anonymous function you can pass around as a value. The reason lambdas exist is to enable a functional, behavior-passing style: instead of writing a whole anonymous inner class to supply one method, you write just the parameters and body.

A lambda's target type is inferred from context — it must be assignable to some functional interface. The JDK ships a standard toolkit in `java.util.function`: `Function<T,R>`, `Predicate<T>`, `Supplier<T>`, `Consumer<T>`, and `BiFunction`, among others. **Method references** (`String::length`, `System.out::println`) are an even shorter form when the lambda just calls an existing method.

Unlike anonymous inner classes, a lambda does **not introduce a new scope**: `this` inside a lambda refers to the *enclosing* instance, not the lambda itself. Lambdas capture local variables that are **effectively final** — you cannot mutate a captured local, which keeps the closure safe to pass across threads.

```
@FunctionalInterface
interface Transformer { String apply(String s); }

Transformer up = s -> s.toUpperCase();           // lambda
Transformer up2 = String::toUpperCase;          // method reference

Predicate<String> nonEmpty = s -> !s.isEmpty();
list.removeIf(nonEmpty.negate());
```

GOTCHA

An interface can still be functional while declaring `default` / `static` methods and re-declaring `public` methods of `Object` (like `equals`) — those don't count toward the "single abstract method" rule. Only genuinely abstract methods count.

37 The Stream API

JAVA 8+

Q How does the Stream API work — intermediate vs terminal operations and laziness?

A A `Stream` is a pipeline for processing a sequence of elements in a **declarative** style — you describe *what* transformation you want, not the loop mechanics. Crucially a stream is **not a data structure**; it holds no storage, does not modify its source, and is consumable only once. Operations split into two kinds, and the distinction is the key to how streams execute.

Intermediate operations (`filter` , `map` , `sorted` , `distinct`) return a new stream and are **lazy** — they build up a plan but do no work when called. A **terminal operation** (`collect` , `forEach` , `reduce` , `count` , `findFirst`) triggers execution and produces a result or side effect. Without a terminal operation, nothing runs at all.

Laziness enables two important optimizations. **Fusion**: the pipeline processes each element through all stages in a single pass rather than materializing an intermediate collection per stage. And **short-circuiting**: operations like `findFirst` , `anyMatch` , and `limit` can stop as soon as the answer is known, which even lets streams work over infinite sources like `Stream.iterate` .

```
List<String> result = names.stream()
    .filter(s -> s.length() > 3)    // lazy
    .map(String::toUpperCase)       // lazy
    .sorted()                       // lazy
    .collect(Collectors.toList()); // TERMINAL: runs it all

// short-circuits: stops at the first match
Optional<String> first = names.stream().filter(pred).findFirst();
```

GOTCHA

Streams are single-use — reusing one after a terminal op throws

`IllegalStateException`. And stream lambdas should be **stateless and side-effect-free**; mutating external state inside `map` / `filter` breaks badly under `parallelStream()`. Use `reduce` / `collect` to accumulate, not a shared variable.

38 Optional

JAVA 8+

Q What is Optional, and what are its proper uses and anti-patterns?

A `Optional<T>` is a container that either holds a value or is empty. Its purpose is to make the **possible absence of a value explicit in the type system**, so a method that might not return a result declares `Optional<User>` instead of returning `null` and hoping the caller checks. This turns silent `NullPointerException`s into a visible contract the compiler and reader can see.

Use it as a **return type** for methods whose result may legitimately be absent, and consume it functionally: `map` / `flatMap` to transform, `filter` to conditionally keep, and crucially `orElse`, `orElseGet`, or `orElseThrow` to supply a fallback or fail loudly. This lets you compose null-safe chains without nested null checks.

The anti-patterns are just as important. Do **not** use `Optional` for fields or method parameters — it adds allocation and awkwardness for no gain; use overloads or nullable params instead. Never call `get()` without first checking `isPresent()` — an unguarded `get()` on empty throws and defeats the entire purpose. And never return `null` from a method whose return type is `Optional`; return `Optional.empty()`.

```
// good: explicit absence, functional fallback
String city = repo.findUser(id)
    .map(User::address)
    .map(Address::city)
    .orElse("unknown");

// bad: reintroduces the NPE you were avoiding
User u = repo.findUser(id).get(); // throws if empty
```

GOTCHA

Prefer `orElseGet(supplier)` over `orElse(value)` when the default is expensive to build: `orElse`'s argument is **always evaluated**, even when the Optional is present, so `orElse(expensiveCall())` runs the call every time regardless.

39 Records

MODERN JAVA

Q What are records and why do they exist?

A Records (standardized in Java 16) are a concise way to declare **immutable data-carrier classes**. They exist to eliminate the boilerplate of the classic value class — the private final fields, constructor, getters, `equals`, `hashCode`, and `toString` that you would otherwise write by hand and keep in sync. From the single header `record Point(int x, int y) {}`, the compiler generates all of it.

A record's declared components become **private final fields** with public *accessor* methods named after the component (`x()` , not `getX()`), a canonical constructor, and value-based `equals` / `hashCode` (comparing all components) plus a readable `toString` . This makes records perfect for DTOs, map keys, multiple return values, and any transparent, shallowly immutable aggregate.

Records carry constraints that flow from being data carriers: they are **implicitly final**, cannot extend another class (they already extend `java.lang.Record`), and cannot declare instance fields beyond the components. You *can* add methods, static members, implement interfaces, and use a **compact canonical constructor** to validate or normalize inputs.

```
record Range(int low, int high) {  
    Range {                                // compact constructor: validate  
        if (low > high)  
            throw new IllegalArgumentException("low > high");  
    }  
}  
Range r = new Range(1, 5);  
r.low();                                  // accessor, not getLow()
```

GOTCHA

Records are only **shallowly immutable**. A `record Team(List<String> members)` still lets callers mutate the passed list, since the field just holds the reference. For true immutability, defensively copy in the compact constructor (e.g., `List.copyOf(members)`).

40 Sealed Classes & Pattern Matching

MODERN JAVA

Q What are sealed classes and how do they combine with pattern matching?

A **Sealed classes/interfaces** (Java 17) let a type **restrict which classes may extend or implement it** via a `permits` clause. The motivation is to model a *closed set* of alternatives — an algebraic "sum type" — so the author controls the full hierarchy. This gives both stronger design intent than a plain abstract class and, critically, lets the compiler **know every possible subtype**.

Each permitted subtype must itself be `final`, `sealed`, or `non-sealed` (which reopens it). Sealing pairs naturally with **pattern matching**. `instanceof` patterns bind and cast in one step (`if (o instanceof String s)`), and **pattern matching for switch** (standard in Java 21) lets you branch on the runtime type, with **record deconstruction patterns** to pull components out directly.

The payoff of combining them is **exhaustiveness checking**: because a sealed type's permitted subtypes are known, a `switch` covering them all needs **no default branch**, and if you later add a new subtype the switch stops compiling until you handle it. This turns "did I handle every case?" from a runtime hope into a compile-time guarantee.

```
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double s) implements Shape {}

double area = switch (shape) {           // no default needed
    case Circle(double r) -> Math.PI * r * r; // deconstruct
    case Square(double s) -> s * s;
};
```

GOTCHA

Order matters in a pattern switch: a more general pattern before a specific one is a **compile error** ("label dominated"). Also handle `null` explicitly — a traditional `switch` throws NPE on null, but a pattern switch can include a `case null ->` label to deal with it cleanly.

41 What is Spring & why use it

SPRING

CORE

Q What is the Spring Framework and why would you use it?

A Spring is a modular application framework whose heart is an **Inversion of Control container**: instead of your objects creating and wiring their own collaborators, the container instantiates them, injects their dependencies, and manages their lifecycle. This decouples *what* a class does from *how* it obtains what it needs, which is the WHY behind almost every Spring feature.

Around that core, Spring layers cross-cutting infrastructure so you write business code, not plumbing: declarative transactions, AOP, data access templates (JDBC, JPA), a web MVC/WebFlux stack, validation, messaging, testing support, and integration with countless third parties. Because collaborators are injected, code becomes easy to unit test with mocks and easy to reconfigure without recompiling.

Spring 6.x targets **Java 17+** and Jakarta EE 9+ (the **javax.*** to **jakarta.*** namespace move), and adds first-class support for GraalVM native images (AOT processing) and observability via Micrometer.

GOTCHA

Spring the *framework* is not Spring Boot. Boot is an opinionated layer on top (auto-configuration, starters, embedded servers) — the IoC container and DI belong to core Spring and work without Boot.

42 Inversion of Control

SPRING

IOC

Q Explain Inversion of Control (IoC).

A IoC is a design principle where control over object creation and flow is inverted from your code to a framework or container. Traditionally an object does `new Dependency()` and controls its own collaborators; with IoC the **container owns that responsibility** and hands finished, wired objects to you. The WHY: this removes hard-coded coupling so components depend on abstractions, not concrete construction logic.

Dependency Injection is the most common implementation of IoC, but the two are not synonyms — the Service Locator pattern is also a form of IoC. In Spring, the `ApplicationContext` is the IoC container: it reads bean definitions, resolves the dependency graph, and injects collaborators.

```
// Without IoC: the class controls its dependency
class OrderService {
    private Repo repo = new JdbcRepo(); // tight coupling
}
// With IoC: the container supplies it
class OrderService {
    private final Repo repo;
    OrderService(Repo repo) { this.repo = repo; }
}
```

GOTCHA

"IoC container" and "DI container" describe the same Spring object from two angles. Interviewers like to hear that DI is a *subset* of IoC, not an equal.

43 Constructor vs setter vs field DI

SPRING

DI

Q Compare constructor, setter, and field injection — which should you prefer?

A Constructor injection is the recommended default. The WHY: mandatory dependencies are supplied at construction, so the object is never in a partially-initialized state, the fields can be `final` (immutable, thread-safe), and the class is trivially testable with plain `new` and no container. Since Spring 4.3, if a class has a single constructor, `@Autowired` is optional.

Setter injection suits genuinely optional or reconfigurable dependencies. Field injection (`@Autowired` on a field) is concise but discouraged: it needs reflection, hides dependencies, cannot produce `final` fields, and makes the class hard to instantiate in a unit test without a container.

```
@Service
public class OrderService {
    private final Repo repo;
    // single constructor - no @Autowired needed
    public OrderService(Repo repo) { this.repo = repo; }
}
```

GOTCHA

Constructor injection also surfaces circular dependencies as a startup failure instead of hiding them — a feature, not a bug. Field injection would silently defer the problem.

44 ApplicationContext vs BeanFactory

SPRING

CONTAINER

Q What is the difference between ApplicationContext and BeanFactory?

A Both are Spring IoC containers, and `ApplicationContext` extends `BeanFactory`. `BeanFactory` is the minimal, lazy container providing basic DI. `ApplicationContext` is the **enterprise superset** you almost always use — it adds event publishing, internationalization (`MessageSource`), automatic `BeanPostProcessor` / `BeanFactoryPostProcessor` registration, environment/property resolution, and AOP integration.

A key behavioral difference: `ApplicationContext` **eagerly pre-instantiates singletons** at startup, so misconfiguration fails fast; a bare `BeanFactory` creates beans lazily on first request. The WHY of preferring ApplicationContext is you get fail-fast startup and all the container callbacks wired automatically.

```
// Common concrete contexts
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);
Repo repo = ctx.getBean(Repo.class);
```

GOTCHA

You rarely touch `BeanFactory` directly today. Naming it as "the parent interface, lazy, no enterprise features" is enough — but know that annotation processors like `@Autowired` require the post-processors that only ApplicationContext auto-registers.

45 What is a Spring bean

SPRING

BEANS

Q What is a Spring bean and how are beans defined?

A A Spring bean is simply **any object instantiated, assembled, and managed by the IoC container**. There is nothing special about the class itself — a plain POJO becomes a bean when the container has a *bean definition* for it (metadata describing its class, scope, dependencies, and lifecycle callbacks). The WHY: the container, not you, controls its creation and wiring.

Beans are defined in three main ways: stereotype/component annotations picked up by scanning (`@Component` , `@Service` , etc.); factory methods in a configuration class (`@Bean`); or legacy XML `<bean>` elements. Annotation-based Java config is the modern idiom.

```
@Configuration
public class AppConfig {
    @Bean
    public Repo repo() { return new JdbcRepo(); }
}
```

Each bean also has an identity (its name/id, defaulting to the decapitalized class or method name) used for lookup and qualifier resolution.

46 Bean scopes

SPRING

SCOPES

Q What bean scopes does Spring provide?

A Scope defines the lifecycle and visibility of a bean instance. **Singleton is the default**: one shared instance per container. The WHY singleton is default: most service/repository beans are stateless, so sharing one instance is cheapest and thread-safe as long as you keep no mutable per-request state.

prototype creates a new instance on every request/injection. Web-aware scopes exist in a web context: **request** (one per HTTP request), **session** (one per HTTP session), plus **application** and **websocket**.

```
@Component
@Scope("prototype")
public class Task { }
```

GOTCHA

Injecting a prototype into a singleton gives you *one* prototype captured at creation — Spring does not create a fresh one per call. Use **ObjectProvider<Task>**, **@Lookup**, or a scoped proxy (**proxyMode**) to get a new instance each time.

47 Bean lifecycle callbacks

SPRING

LIFECYCLE

Q Describe the bean lifecycle and its callback hooks.

A For a singleton: the container instantiates the bean, populates its dependencies, invokes `*Aware` callbacks, runs

`BeanPostProcessor.postProcessBeforeInitialization`, then **initialization callbacks**, then post-processors' `after` phase (where AOP proxies are applied), leaving a ready bean. On shutdown it runs destruction callbacks. The WHY of hooks: you get deterministic points to acquire/release resources after injection is complete.

Three ways to hook init/destroy, in order of preference: JSR-250

`@PostConstruct` / `@PreDestroy` (in Spring 6 these live in `jakarta.annotation`); implementing `InitializingBean` / `DisposableBean` (couples you to Spring); or `@Bean(initMethod=..., destroyMethod=...)`.

```
@Component
public class Cache {
    @PostConstruct
    void warmUp() { /* deps are ready here */ }
    @PreDestroy
    void flush() { }
}
```

GOTCHA

`@PreDestroy` does *not* run for prototype beans — Spring stops managing them after creation, so you must dispose them yourself. It also won't run if the JVM is killed without a clean context close.

48 @Component vs @Bean

SPRING

CONFIG

Q What is the difference between @Component and @Bean?

A Both register a bean, but at different levels. `@Component` (and its stereotypes) is a **class-level marker** discovered by component scanning — Spring constructs the instance for you. `@Bean` is a **method-level** annotation inside a `@Configuration` class where *you* write the factory method and return the instance. The WHY of choosing: use `@Component` for your own classes; use `@Bean` when you need programmatic control or when the type is third-party code you cannot annotate.

```
// You own the class -> stereotype
@Service class OrderService { }

// Third-party type -> factory method
@Configuration
class Cfg {
    @Bean
    ObjectMapper mapper() { return new ObjectMapper(); }
}
```

GOTCHA

`@Bean` gives you the constructor call, so you can pass arguments, apply conditional logic, or return different implementations. `@Component` cannot — it relies purely on the container invoking a constructor.

49 @Configuration & proxying

SPRING

CONFIG

Q How do @Configuration classes work, and what is the proxying about?

A A `@Configuration` class is a source of bean definitions via its `@Bean` methods. What makes it special is **"full" mode**: Spring wraps the config class in a CGLIB subclass proxy that intercepts inter-bean method calls. The WHY: if one `@Bean` method calls another, the proxy returns the *shared singleton* from the container rather than executing the method again and creating a duplicate.

```
@Configuration
class Cfg {
    @Bean Repo repo() { return new JdbcRepo(); }
    @Bean Service svc() {
        // returns the SAME repo singleton, not a new one
        return new Service(repo());
    }
}
```

Setting `@Configuration(proxyBeanMethods = false)` ("lite" mode) skips the proxy for a small startup/native-image gain, but then each method call constructs a new object — safe only if methods don't call each other. Plain `@Component` classes with `@Bean` methods are always lite mode.

GOTCHA

Because the proxy is a CGLIB subclass, a `@Configuration` class cannot be `final` and its `@Bean` methods cannot be `final` or `private`.

50 Component scanning

SPRING

SCANNING

Q What is component scanning and how is it configured?

A Component scanning is how Spring **automatically discovers** classes annotated with `@Component` (or stereotypes `@Service`, `@Repository`, `@Controller`, `@Configuration`) on the classpath and registers them as beans. The WHY: it removes the need to declare every bean by hand. You enable it with `@ComponentScan`, which defaults to scanning the package of the annotated class and all sub-packages.

```
@Configuration
@ComponentScan(basePackages = "com.acme.app")
public class AppConfig { }
```

You can narrow or widen discovery with `includeFilters` / `excludeFilters` (by annotation, assignable type, regex, or custom). In Spring Boot, `@SpringBootApplication` bundles a `@ComponentScan` rooted at the main class's package — which is why placement of that class matters.

GOTCHA

Put your main/config class in a *root* package. If it sits in a deep sub-package, components in sibling packages won't be scanned and you'll get baffling "no qualifying bean" errors.

51 @Autowired resolution

SPRING

DI

Q How does @Autowired resolve dependencies?

A `@Autowired` tells Spring to inject a collaborator. Resolution is **by type first**: the container finds a bean assignable to the target type. The WHY of the by-type default: it decouples you from bean names. If exactly one candidate matches, it's injected; if none match and the dependency is required, startup fails.

When multiple beans match the type, Spring falls back to **by name** — matching the field/parameter name to a bean name — and honors `@Primary` and `@Qualifier`. If it still can't disambiguate, it throws `NoUniqueBeanDefinitionException`.

```
@Service
class Report {
    private final DataSource ds;
    @Autowired // optional on a sole constructor
    Report(DataSource ds) { this.ds = ds; }
}
```

GOTCHA

Injecting a collection type, e.g. `List<Validator>`, is not ambiguous — Spring injects *all* matching beans. And `@Autowired(required = false)` or `Optional<T>` lets a missing dependency resolve to null/empty instead of failing.

52 @Qualifier & @Primary

SPRING

DI

Q How do @Qualifier and @Primary resolve ambiguity?

A When several beans satisfy a type, Spring needs a tie-breaker. `@Primary` marks **one bean as the default** winner whenever no more specific hint is given — a broad, "usually this one" choice. `@Qualifier` is the **precise, per-injection** selector naming exactly which bean to use. The WHY of having both: `@Primary` sets a sensible default centrally, while `@Qualifier` overrides it locally.

```
@Bean @Primary PaymentGateway stripe() { ... }
@Bean @Qualifier("legacy") PaymentGateway paypal() { ... }

@Service
class Checkout {
    Checkout(@Qualifier("legacy") PaymentGateway g) { ... }
}
```

Qualifier beats primary when both apply. You can also create custom qualifier annotations for type-safe selection instead of string names.

GOTCHA

If two beans are both `@Primary` for the same type, you're back to `NoUniqueBeanDefinitionException`. There must be at most one primary per type.

53 Circular dependencies

SPRING

DI

Q What causes circular dependencies and how do you handle them?

A A circular dependency is when bean A needs B and B needs A (directly or via a chain). With **constructor injection on both sides**, Spring cannot construct either first, so it throws `BeanCurrentlyInCreationException` at startup. The WHY it fails: neither object can exist without the other already existing — a genuine chicken-and-egg.

Spring can resolve *setter/field* cycles among singletons using an early-reference cache (it exposes a half-built bean), but that's a crutch. The real fix is usually to break the cycle: extract a third collaborator, use an event or an interface, or make one side lazy with `@Lazy` so a proxy is injected and the real bean is resolved on first use.

```
@Service
class A {
    A(@Lazy B b) { ... } // injects a proxy, breaks the cycle
}
```

GOTCHA

Spring Boot 2.6+ *disables* automatic circular-reference resolution by default; you must set `spring.main.allow-circular-references=true` to re-enable it — a nudge to redesign rather than paper over the cycle.

54 @Value & properties

SPRING

CONFIG

Q How do you inject external properties with @Value?

A `@Value` injects a resolved value — a literal, a property, or a SpEL expression — into a field or parameter. Property placeholders use `${...}` and are resolved against the Spring `Environment` (property files, environment variables, command line). The WHY: it externalizes configuration so the same build runs in different environments.

```
@Component
class Mailer {
    @Value("${smtp.host:localhost}") // default after colon
    private String host;
    @Value("#{systemProperties['user.timezone']}") // SpEL
    private String tz;
}
```

Note the syntax difference: `${...}` is a *property placeholder*, while `#{...}` is a *SpEL expression*. For grouping many related keys, prefer type-safe

`@ConfigurationProperties` classes over scattering `@Value` annotations.

GOTCHA

Providing a default with `${key:default}` avoids startup failure when the key is missing. Without a default and without the property present, you get an

`IllegalArgumentException` resolving the placeholder.

55 Spring AOP core concepts

SPRING

AOP

Q Explain the core concepts of Spring AOP.

A AOP modularizes **cross-cutting concerns** — logging, security, transactions — that would otherwise be scattered across many methods. The WHY: you write the concern once as an *aspect* and declaratively apply it, keeping business code clean. Key terms:

Join point: a point in program execution where advice can run (in Spring AOP, always a method execution). **Pointcut:** an expression selecting which join points match.

Advice: the action taken at a matched join point. **Aspect:** the module bundling pointcuts and advice. **Weaving:** linking aspects into target objects — Spring does this at runtime via proxies.

```
@Aspect
@Component
class LoggingAspect {
    @Before("execution(* com.acme.service.*.*(..))")
    void log(JoinPoint jp) {
        // runs before every service method
    }
}
```

GOTCHA

Spring AOP is *proxy-based and method-execution only* — it cannot advise field access or constructor calls. For those you need full AspectJ with compile-time or load-time weaving.

56 Types of advice

SPRING

AOP

Q What advice types does Spring AOP support?

A There are five, distinguished by *when* they fire relative to the join point. `@Before` runs before the method. `@AfterReturning` runs after a normal return (and can read the returned value). `@AfterThrowing` runs when the method throws. `@After` is a finally-style callback that runs regardless of outcome. `@Around` is the most powerful: it wraps the call and decides whether/when to proceed.

```
@Around("execution(* *..*Service.*(..))")
public Object time(ProceedingJoinPoint pjp) throws Throwable {
    long t = System.nanoTime();
    try {
        return pjp.proceed(); // must call to continue
    } finally {
        record(System.nanoTime() - t);
    }
}
```

The WHY of `@Around`: because it receives a `ProceedingJoinPoint`, it can short-circuit, retry, transform arguments, or replace the return value — everything the others can, combined.

GOTCHA

An `@Around` advice that forgets to call `proceed()` silently swallows the target method — it never runs, and the caller gets `null`. Always return the result of `proceed()`.

57 JDK proxy vs CGLIB

SPRING

AOP

Q How does Spring's proxy mechanism choose JDK dynamic proxy vs CGLIB?

A Spring implements AOP by creating a **proxy that wraps the target bean** and inserts advice around method calls. Two strategies: **JDK dynamic proxies** implement the target's interfaces (the proxy is a sibling type, not a subclass); **CGLIB** generates a runtime *subclass* and overrides methods. The WHY of two mechanisms: JDK proxies need an interface, so CGLIB covers concrete classes that have none.

Historically Spring used a JDK proxy when the bean implemented at least one interface and CGLIB otherwise. Modern Spring Boot **defaults to CGLIB for everything** (`proxyTargetClass = true`) to avoid the "must inject by interface" surprise.

```
// Force CGLIB even for interface-based beans
@EnableAspectJAutoProxy(proxyTargetClass = true)
```

GOTCHA

With a JDK proxy you must inject by the *interface*, not the concrete class, or you get a `ClassCastException`. CGLIB can't proxy `final` classes or override `final` / `private` methods, so advice on those silently won't apply.

58 @Transactional internals

SPRING

TRANSACTIONS

Q What does @Transactional do and how does it work?

A `@Transactional` declares that a method runs inside a database transaction: Spring begins a transaction before the method, **commits on normal return and rolls back on a runtime exception**. The WHY: you get declarative, boilerplate-free transaction management instead of manual begin/commit/rollback.

It is implemented with **the same AOP proxy mechanism**. A proxy intercepts the call, obtains a connection from the `PlatformTransactionManager`, binds it to the thread, invokes your method, then commits or rolls back. Because it's proxy-based, the advice only triggers when the call arrives *through the proxy*.

```
@Service
class BankService {
    @Transactional
    public void transfer(Long from, Long to, BigDecimal amt) {
        debit(from, amt);
        credit(to, amt); // both commit or both roll back
    }
}
```

GOTCHA

Two classic traps: (1) a *self-invocation* — one method in a bean calling another `@Transactional` method directly — bypasses the proxy, so no transaction starts. (2) By default only *unchecked* exceptions roll back; a checked exception commits unless you set `rollbackFor`.

59 Stereotype annotations

SPRING

STEREOTYPES

Q What are `@Service`, `@Repository`, and `@Controller`, and how do they differ from `@Component`?

A All are **specializations of `@Component`** — they are themselves meta-annotated with it, so scanning picks them up identically. The WHY they exist: they express *role/intent* (semantic clarity) and are hooks for role-specific behavior, even though wiring-wise they're the same.

`@Repository` is the one with real added behavior: it enables **persistence exception translation**, converting vendor-specific JDBC/JPA exceptions into Spring's consistent `DataAccessException` hierarchy. `@Service` marks the business/service layer (no extra behavior today, purely semantic). `@Controller` (and `@RestController`) marks web handlers and is scanned by Spring MVC for request mapping.

```
@Repository class JpaUserRepo { } // exception translation
@Service    class UserService { } // business layer
@Controller class UserController { } // web layer
```

GOTCHA

Functionally you *could* use bare `@Component` everywhere and it would still wire correctly — but you'd lose `@Repository`'s exception translation and the layered readability interviewers look for.

60 Spring vs Spring Boot

SPRING

BOOT

Q What is the difference between the Spring Framework and Spring Boot?

A Spring Framework is the **foundational toolkit** (IoC container, DI, AOP, MVC, data access). It is powerful but historically required significant manual configuration and dependency curation. Spring Boot is an **opinionated layer built on top** of it. The WHY of Boot: eliminate boilerplate configuration so you can run a production-grade app in minutes.

Boot's headline features: **auto-configuration** (it inspects the classpath and configures sensible beans automatically), **starter dependencies** (curated, version-aligned dependency bundles like `spring-boot-starter-web`), **embedded servers** (Tomcat/Jetty/Netty bundled, so you run a plain JAR), and production tooling via Actuator (health, metrics).

```
@SpringBootApplication // = @Configuration + @EnableAutoConfiguration +
@ComponentScan
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

GOTCHA

Boot doesn't replace or hide Spring — it *configures* it. Everything Boot does you could do by hand in plain Spring; understanding that distinction (and how to override an auto-configuration) is what separates users from experts.

61 What Spring Boot Is

SPRING BOOT

Q What is Spring Boot and which problems does it solve?

A Spring Boot is an opinionated layer on top of the Spring Framework that makes it fast to build stand-alone, production-grade applications with **minimal configuration**. The core problem it solves is the historical *configuration burden* of plain Spring: XML files, manual bean wiring, hand-picked dependency versions, and an external servlet container to deploy a WAR into.

Boot tackles this with four pillars. **Auto-configuration** wires sensible beans based on what is on the classpath. **Starters** are curated dependency bundles so you add one artifact instead of ten. An **embedded server** lets you run `java -jar app.jar` with no external Tomcat. And **Actuator** gives you health, metrics, and management endpoints out of the box.

The guiding philosophy is **"convention over configuration"**: Boot supplies working defaults you can override at any point. It does *not* replace Spring or generate code — it configures the same Spring you already know, just automatically.

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

GOTCHA

Boot is not "a faster Spring" or a new framework — it is Spring plus automation. Saying it "replaces the Spring Framework" is a red flag in interviews.

62 Auto-Configuration

SPRING BOOT

Q What is auto-configuration and how does it work internally?

A Auto-configuration is the mechanism by which Boot inspects the classpath, existing beans, and properties, then conditionally registers beans you would otherwise define yourself. The *why*: if `spring-webmvc` and Tomcat are present you almost certainly want a `DispatcherServlet` and an embedded server, so Boot creates them unless you say otherwise.

It is triggered by `@EnableAutoConfiguration` (folded into `@SpringBootApplication`). At startup Boot reads auto-configuration class names from every jar. In Boot **2.7+** and all of 3.x these live in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` — one fully-qualified class name per line. The legacy `spring.factories` key is deprecated for this purpose.

Each listed class is a `@Configuration` guarded by `@Conditional` annotations (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, etc.). Conditions are evaluated in order, and user-defined beans win because most auto-config backs off via `@ConditionalOnMissingBean`. Run with `--debug` to print the **condition evaluation report** showing what matched and why.

```
# META-INF/spring/...AutoConfiguration.imports
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration
```

GOTCHA

Auto-config classes run *after* your own `@Configuration`, so your beans exist first and `@ConditionalOnMissingBean` correctly backs off. Reverse that ordering assumption and the logic makes no sense.

63 Starters

SPRING BOOT

Q What are Spring Boot starters?

A Starters are curated, versioned dependency descriptors. Adding `spring-boot-starter-web` transitively pulls Spring MVC, Jackson, validation, and embedded Tomcat — all at mutually compatible versions. The problem they solve is **dependency hell**: you no longer hand-pick and align dozens of library versions.

A starter is usually an almost-empty pom whose value is its transitive dependency list. Names follow a convention: official ones are `spring-boot-starter-*`; third-party ones must use their own prefix, e.g. `mybatis-spring-boot-starter` (Spring reserves the `spring-boot-starter-` prefix).

Common ones: `-web` (MVC + Tomcat), `-webflux` (reactive + Netty), `-data-jpa`, `-security`, `-actuator`, `-test` (JUnit 5, Mockito, AssertJ). Versions come from the parent BOM, so you typically omit `<version>`.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

GOTCHA

Want Jetty instead of Tomcat? You must *exclude* `spring-boot-starter-tomcat` from the web starter and add `spring-boot-starter-jetty` — simply adding Jetty leaves both on the classpath and startup fails.

64 @SpringBootApplication

SPRING BOOT

Q Break down @SpringBootApplication into its component annotations.

A It is a convenience meta-annotation combining three annotations you would otherwise stack manually. Understanding the split matters because each has knobs (like base packages or excludes) you may need to reach.

`@SpringBootConfiguration` is a specialization of `@Configuration`, marking the class as a source of bean definitions (and letting test slices find the primary config).

`@EnableAutoConfiguration` switches on the auto-configuration machinery described earlier. `@ComponentScan` scans the annotated class's package and its sub-packages for `@Component`, `@Service`, `@Repository`, and `@Controller` beans.

Because component scanning starts from the main class's package, the **placement of the main class is significant**: put it in a root package above your feature packages.

You can customize via attributes such as `scanBasePackages` or `exclude`.

```
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public class App { } // == @SpringBootApplication
```

GOTCHA

If beans in a sibling package are "not found", the main class is probably too deep. Components outside the main class's package tree are never scanned unless you widen `scanBasePackages`.

65 Properties vs YAML

SPRING BOOT

Q application.properties vs application.yml — differences and trade-offs?

A Both configure the same `Environment`; they are just formats. `.properties` is flat `key=value`, while `.yml` is hierarchical and less repetitive, which matters for deeply nested config. The *why* to prefer YAML: readability for structured trees and native list/map syntax.

YAML supports multiple documents in one file separated by `---`, making profile-specific blocks compact. Its downside is **whitespace sensitivity** and a few surprises (unquoted `yes` / `no` / `on` historically parsed as booleans). Properties files avoid indentation bugs and support easy override via command line.

If both files exist in the same location, `.properties` takes precedence over `.yml` for the same key. YAML also cannot be loaded by `@PropertySource`, which only reads properties files.

```
# application.yml
server:
  port: 8081
spring:
  datasource:
    url: jdbc:postgresql://localhost/app
# --- separates profile documents ---
```

GOTCHA

`@PropertySource("classpath:foo.yml")` silently loads nothing — it only handles `.properties`. For custom YAML you need a `PropertySourceFactory`.

66 Profiles

SPRING BOOT

Q What are Spring profiles and how do you use them?

A Profiles let you register beans and load configuration conditionally per environment (dev, test, prod). The *why*: one artifact, many environments — you flip a switch rather than rebuilding. A bean or config class annotated `@Profile("prod")` is only created when that profile is active.

Activate profiles with `spring.profiles.active`, set via property, env var `SPRING_PROFILES_ACTIVE`, or `--spring.profiles.active=prod`. Boot then also loads profile-specific files like `application-prod.yml` layered on top of the base `application.yml`. Multiple profiles can be active at once.

Boot 3 uses **profile groups** (`spring.profiles.group`) to activate several profiles together, and `spring.config.activate.on-profile` inside a document to gate a config block. Note `spring.profiles.active` cannot itself be set from a profile-specific file.

```
@Configuration
@Profile("prod")
public class ProdDataSourceConfig { }
```

GOTCHA

If no profile is set, only the `default` profile runs. Beans marked `@Profile("dev")` silently do not exist in prod — a frequent cause of "works on my machine" missing-bean errors.

67 Config Precedence

SPRING BOOT

Q Explain externalized configuration and property source precedence.

A Externalized configuration means the same jar can be tuned from outside without rebuilding — via files, environment variables, system properties, or command-line arguments. Boot merges many **property sources** into one **Environment**, and precedence decides who wins on a key clash.

From *highest* to lowest (abridged, Boot 3): devtools global settings;

@TestPropertySource; command-line arguments; **SPRING_APPLICATION_JSON**; servlet params; JNDI; Java System properties; OS environment variables; profile-specific application files outside the jar, then inside; plain application files outside the jar, then inside; **@PropertySource**; finally **SpringApplication** default properties.

The practical rule: **more specific and more external beats more general and more embedded**. So a command-line **--server.port=9000** overrides anything in **application.yml**, and an env var overrides a packaged file. Relaxed binding maps **SERVER_PORT** to **server.port**.

```
# env var overrides packaged application.yml
SERVER_PORT=9000 java -jar app.jar --spring.profiles.active=prod
```

GOTCHA

Command-line args beat env vars and files but can be disabled with

setAddCommandLineProperties(false). Also, **application.properties** outside the jar (same dir) overrides the one packaged inside.

68 @ConfigurationProperties vs @Value

SPRING BOOT

Q @ConfigurationProperties vs @Value — when to use which?

A `@Value` injects a single property with SpEL support into a field or parameter.

`@ConfigurationProperties` binds a whole tree of properties onto a typed POJO. The *why*: for grouped, structured config, type-safe binding is more maintainable and supports validation, relaxed binding, and metadata.

`@ConfigurationProperties` supports **relaxed binding** (`my-prop` , `myProp` , `MY_PROP` all match), nested objects, lists, maps, and JSR-303 validation via `@Validated` . `@Value` does not do relaxed binding and requires exact keys, but it evaluates SpEL expressions like `#{...}` , which `@ConfigurationProperties` cannot.

Register the properties class with `@EnableConfigurationProperties` or `@ConfigurationPropertiesScan` . Boot 3 favors immutable constructor binding, letting you use `record` types. Reach for `@Value` only for a one-off value; use `@ConfigurationProperties` for anything grouped.

```
@ConfigurationProperties(prefix = "app.mail")
public record MailProps(String host, int port) { }
```

GOTCHA

Constructor-bound (record) properties classes cannot also be annotated `@Component` ; register them with `@EnableConfigurationProperties(MailProps.class)` or `@ConfigurationPropertiesScan` instead.

69 Embedded Servers

SPRING BOOT

Q How do embedded servers work and how do you change or configure them?

A Boot embeds a servlet (or reactive) container inside the jar, so the application boots the server rather than being deployed into one. This is why `java -jar` just works and why each service can own its container version. Tomcat is the default for `-web`; alternatives are Jetty and Undertow (Netty for WebFlux).

To switch, exclude `spring-boot-starter-tomcat` and add `spring-boot-starter-jetty` or `-undertow`. Common tuning uses `server.*` properties: `server.port`, `server.servlet.context-path`, `server.tomcat.threads.max`, `server.compression.enabled`, and TLS via `server.ssl.*`.

For programmatic control, implement

`WebServerFactoryCustomizer<ConfigurableServletWebServerFactory>`. Setting `server.port=0` picks a random free port (great for tests), and `-1` disables the web server. You can still build a WAR for a traditional container by extending `SpringBootServletInitializer`.

```
@Component
class PortCustomizer implements
    WebServerFactoryCustomizer<ConfigurableServletWebServerFactory> {
    @Override public void customize(ConfigurableServletWebServerFactory f) {
        f.setPort(8085);
    }
}
```

GOTCHA

`server.port=0` means "random port", not "disabled". To turn the web server off entirely set `spring.main.web-application-type=none`.

70 Actuator

SPRING BOOT

Q What is Spring Boot Actuator and what does it provide?

A Actuator adds production-ready monitoring and management endpoints over HTTP or JMX with almost no code. The *why*: operability — health checks for orchestrators, metrics for dashboards, and introspection of beans, config, and mappings, all standardized.

Add `spring-boot-starter-actuator` and you get endpoints like `/actuator/health`, `/actuator/info`, `/actuator/metrics`, `/actuator/env`, `/actuator/loggers`, and `/actuator/threaddump`. By default only **health** is exposed over HTTP; others must be opted in via `management.endpoints.web.exposure.include`.

Endpoints can be enabled/disabled individually, moved to a separate management port (`management.server.port`), and secured with Spring Security. `/health` can show details only to authenticated users via `management.endpoint.health.show-details`. Metrics are backed by **Micrometer**, exportable to Prometheus, Datadog, and others.

```
# expose selected endpoints
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: when_authorized
```

GOTCHA

Exposing `*` in production without securing the management endpoints leaks `/env`, `/heapdump`, and `/threaddump` — a real information-disclosure risk.

71 Custom Health & Metrics

SPRING BOOT

Q How do you add custom health indicators and metrics?

A For a custom health check, implement `HealthIndicator` (or `ReactiveHealthIndicator`) and return a `Health` with status and detail data. The *why*: you often need to signal readiness based on a downstream dependency — a queue, a license server — that Boot cannot know about. A bean named `fooHealthIndicator` surfaces under the `foo` key in `/actuator/health`.

The aggregate health is the **worst** status among contributors; any `DOWN` makes the overall status `DOWN` (HTTP 503). Boot 3 also distinguishes `liveness` and `readiness` probes for Kubernetes via health *groups*.

For metrics, inject Micrometer's `MeterRegistry` and register counters, timers, or gauges; or annotate methods with `@Timed`. Custom meters appear under `/actuator/metrics/{name}` and export to whatever backend you configured.

```
@Component
class QueueHealth implements HealthIndicator {
    @Override public Health health() {
        return queueUp() ? Health.up().build()
            : Health.down().withDetail("queue", "unreachable").build();
    }
}
```

GOTCHA

Health details are hidden by default (`show-details: never`). Your carefully populated `withDetail(...)` map will not appear until you set `show-details` to `always` or `when_authorized`.

72 Logging

SPRING BOOT

Q How is logging configured in Spring Boot?

A Boot uses **SLF4J as the facade and Logback as the default implementation**, pre-configured with a sensible console pattern. The *why*: you get working, colored logging with zero setup, yet can retarget or replace it. `spring-boot-starter-logging` is pulled in by every other starter.

Tune levels with `logging.level.<package>=DEBUG`, write to a file with `logging.file.name` or `logging.file.path`, and adjust patterns with `logging.pattern.console`. For full control, drop a `logback-spring.xml` on the classpath — the `-spring` variant supports Boot extensions like `<springProfile>`.

To swap to Log4j2, exclude the default logging starter and add `spring-boot-starter-log4j2`. Levels are also changeable at runtime via the Actuator `/actuator/loggers` endpoint (POST a new level), which is invaluable for debugging production without a restart.

```
# application.yml
logging:
  level:
    org.springframework.web: DEBUG
    com.example: TRACE
  file:
    name: logs/app.log
```

GOTCHA

Use `logback-spring.xml`, not `logback.xml`. Logback reads plain `logback.xml` too early for Boot to inject properties, so `<springProfile>` and property placeholders won't resolve.

73 DevTools

SPRING BOOT

Q What does Spring Boot DevTools provide?

A DevTools is a development-time productivity aid. Its headline feature is **automatic restart**: when classpath files change, it restarts the application context quickly using two classloaders — a base loader for unchanging library classes and a restart loader for your code, so only your classes reload.

It also offers LiveReload (browser refresh on resource change), sensible dev property defaults (disabling template and static-resource caching so edits show immediately), and a global `~/.spring-boot-devtools.properties`. It even enables remote debugging support.

Add `spring-boot-devtools` with `<optional>true</optional>`. Crucially it is *disabled automatically* when the app runs from a fully packaged jar (i.e., in production), so it never ships live. It is not a hot-swap of arbitrary changes — structural changes still need a real restart, which it performs for you.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <optional>true</optional>
</dependency>
```

GOTCHA

DevTools deactivates itself when running from a packaged jar, so relying on its property overrides (like disabled caching) in production is impossible — those defaults only apply in development.

74 Packaging & Fat Jars

SPRING BOOT

Q How is a Spring Boot app packaged and run? What are layered jars?

A The `spring-boot-maven-plugin` (or Gradle plugin) repackages your build into an **executable fat jar** that contains your classes, all dependency jars, and an embedded server. The *why*: a single self-contained artifact you can run anywhere with `java -jar`, no external container.

Boot uses a nested-jar layout with a custom `JarLauncher` declared in the manifest's `Main-Class`; your real main sits in `Start-Class`. Dependencies live under `BOOT-INF/lib` and your code under `BOOT-INF/classes`, loaded by a launcher that reads jars-within-jars.

Layered jars (default since 2.4) split contents into layers — dependencies, spring-boot-loader, snapshot-dependencies, application — ordered by change frequency. Docker builds copy each layer separately so stable dependency layers stay cached across rebuilds, and only the small application layer changes. Extract with `java -Djarmode=layertools -jar app.jar extract`.

```
# Dockerfile using layered jar
COPY --from=builder dependencies/ ./
COPY --from=builder spring-boot-loader/ ./
COPY --from=builder application/ ./
```

GOTCHA

A Boot fat jar is not a normal jar — you cannot just add it to another app's classpath and import its classes, because of the nested `BOOT-INF` layout. For a library module, build a plain jar instead.

75 Runners

SPRING BOOT

Q CommandLineRunner vs ApplicationRunner?

A Both are callbacks executed **once, right after the application context is fully initialized** but before `SpringApplication.run` returns. The *why*: a hook for startup logic — seeding data, warming caches, or running a batch job — with all beans available.

The only difference is the argument type. `CommandLineRunner.run(String... args)` receives the raw command-line strings.

`ApplicationRunner.run(ApplicationArguments args)` receives a parsed abstraction that separates *option* args (`--name=value`) from non-option args and exposes helper methods like `getOptionValues` .

You can define several runners; order them with `@Order` or by implementing `Ordered` . If a runner throws, application startup fails. Prefer `ApplicationRunner` when you need structured access to flags.

```
@Component
class Seeder implements ApplicationRunner {
    @Override public void run(ApplicationArguments args) {
        if (args.containsOption("seed")) seedData();
    }
}
```

GOTCHA

Runners execute on the main startup thread, so slow work delays readiness. An exception thrown from a runner aborts the whole application — wrap risky logic if partial startup is acceptable.

76 Conditional Annotations

SPRING BOOT

Q Explain the conditional annotations (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, etc.).

A These annotations register a bean or configuration only when a condition holds. They are the engine behind auto-configuration's **"back off if the user already did it"** behavior, and you can use them in your own config too. All are built on the general `@Conditional` SPI.

Key ones: `@ConditionalOnClass` / `@ConditionalOnMissingClass` (classpath presence), `@ConditionalOnBean` / `@ConditionalOnMissingBean` (bean presence — the classic override hook), `@ConditionalOnProperty` (a property has a value), `@ConditionalOnWebApplication`, `@ConditionalOnResource`, and `@ConditionalOnExpression` for SpEL.

Bean conditions are evaluated in a specific order and are sensitive to **registration timing**, so they belong on auto-configuration classes (which run last), not on ordinary user config where bean order is unpredictable. The `--debug` report shows the positive and negative matches.

```
@Bean
@ConditionalOnMissingBean
@ConditionalOnProperty(name = "app.cache", havingValue = "true")
public CacheService cacheService() { return new DefaultCacheService(); }
```

GOTCHA

`@ConditionalOnMissingBean` on your own `@Configuration` is unreliable because ordering isn't guaranteed there. It only works dependably in auto-config, which Boot evaluates after all user beans.

77 Excluding Auto-Config

SPRING BOOT

Q How do you customize or exclude auto-configuration?

A Because auto-config is conditional, the primary "customization" is simply *defining your own bean* — most auto-config uses `@ConditionalOnMissingBean` and backs off. This is the idiomatic override: you rarely fight auto-config, you just supply the bean and it steps aside.

To remove an auto-configuration entirely, use `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`, or the string form `excludeName`, or the property `spring.autoconfigure.exclude`. A common case is excluding `DataSourceAutoConfiguration` when there is no database yet, to stop Boot from failing on a missing URL.

You can also influence ordering with `@AutoConfigureBefore` / `@AutoConfigureAfter`, and toggle features via `application.properties`. To see what is active, run with `--debug` for the **condition evaluation report**.

```
@SpringBootApplication(  
    exclude = { DataSourceAutoConfiguration.class }  
)  
public class App { }  
# or: spring.autoconfigure.exclude=...DataSourceAutoConfiguration
```

GOTCHA

You can only `exclude` a class that is actually an auto-configuration; excluding a regular `@Configuration` throws an error. Match the exact class name that appears in `AutoConfiguration.imports`.

78 SpringApplication Startup

SPRING BOOT

Q How do you customize startup with SpringApplication (banners, listeners)?

A `SpringApplication` drives the bootstrap sequence: it decides the application type, prepares the `Environment`, prints the banner, creates and refreshes the context, and fires lifecycle events. The *why* to use the builder form is fine-grained control the static `run` shortcut hides.

Customize by constructing it directly or via `SpringApplicationBuilder`: set the banner mode (`Banner.Mode.OFF`), disable the command-line properties, add default properties, or set the web application type. A custom banner comes from a `banner.txt` on the classpath.

Subscribe to startup **events** with `ApplicationListener` or by registering a listener before the context exists (for early events like `ApplicationStartingEvent`). The ordered sequence includes `ApplicationStartingEvent`, `ApplicationEnvironmentPreparedEvent`, `ApplicationContextInitializedEvent`, `ApplicationPreparedEvent`, `ApplicationStartedEvent`, `ApplicationReadyEvent`, and on failure `ApplicationFailedEvent`.

```
public static void main(String[] args) {
    SpringApplication app = new SpringApplication(App.class);
    app.setBannerMode(Banner.Mode.OFF);
    app.addListeners(new MyStartupListener());
    app.run(args);
}
```

GOTCHA

To catch very early events like `ApplicationStartingEvent`, an `@Bean` listener is too late — the context doesn't exist yet. Register it via `app.addListeners(...)` or `META-INF/spring.factories`.

79 Parent BOM

SPRING BOOT

Q How does dependency management and the parent BOM work?

A Boot centralizes version management in a **Bill of Materials**. Inheriting from `spring-boot-starter-parent` (or importing `spring-boot-dependencies`) supplies a curated `dependencyManagement` section that pins compatible versions for hundreds of common libraries. The *why*: you omit `<version>` tags and avoid conflicting transitive versions.

`spring-boot-starter-parent` extends `spring-boot-dependencies` and additionally sets Java version, UTF-8 encoding, resource filtering, and plugin configuration (like the repackaging goal). If your project already has a corporate parent, *import* the BOM in `dependencyManagement` with `scope=import` instead of inheriting.

To override a managed version, redefine the property (e.g. `<jackson.version>`) or declare an explicit version — your declaration wins over the BOM. In Gradle the equivalent is the **Spring Boot Gradle plugin** plus the dependency-management plugin (or the platform() BOM).

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.3.0</version>
</parent>
```

GOTCHA

To bump a single library, override the version *property* rather than adding a second dependency declaration — otherwise the BOM still manages the transitive version and you get a mismatch.

80 Spring Boot 3

SPRING BOOT

Q What is new and important in Spring Boot 3?

A Spring Boot 3 (on Spring Framework 6) raised the baseline: **Java 17 minimum** and a migration from the `javax.*` namespace to `jakarta.*` (Jakarta EE 9+). The *why* this matters: every `javax.persistence`, `javax.servlet`, and `javax.validation` import must become `jakarta.*`, and third-party libraries must be Jakarta-compatible.

It brings first-class **GraalVM native image** support via ahead-of-time (AOT) processing: Boot generates reflection/config hints and pre-computes bean definitions at build time, producing fast-starting, low-memory native executables (`spring-boot-starter-parent` + `native` profile / `nativeCompile`). AOT trades some runtime dynamism for startup speed.

Observability is unified through **Micrometer Observation** and Micrometer Tracing (replacing Spring Cloud Sleuth), giving correlated metrics, traces, and logs. Other notables: `RestClient` and HTTP interface clients, problem-detail (RFC 7807) error responses, and the `AutoConfiguration.imports` file replacing `spring.factories` for auto-config.

```
// Boot 2 -> Boot 3
// import javax.persistence.Entity;
import jakarta.persistence.Entity;
```

GOTCHA

The `javax` to `jakarta` switch is source-incompatible: any dependency still emitting `javax.servlet` classes will not work on Boot 3. Check every library for a Jakarta-compatible release before upgrading.

81 Spring Data JPA Overview

SPRING DATA

Q What is Spring Data JPA and what does it give you over plain JPA?

A Spring Data JPA is a layer on top of JPA/Hibernate that removes boilerplate DAO code. The core idea: you declare a *repository interface*, and Spring generates the implementation at runtime as a proxy. You never write "create an EntityManager, begin transaction, run query, map results" plumbing yourself.

It gives you CRUD out of the box, **derived query methods** (parsed from method names), paging and sorting, `@Query` for custom JPQL/native SQL, auditing (`@CreatedDate`), specifications and QueryDSL for dynamic queries, and projections for partial reads. Under the hood it still uses a JPA provider (Hibernate by default in Boot).

The mental model that matters in interviews: Spring Data JPA is **not** a replacement for JPA — it is a productivity layer. Hibernate still manages the persistence context, dirty checking, flushing, and the first-level cache. Understanding that boundary explains most "why did my update save without a save() call?" surprises.

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByLastName(String lastName);
}
```

GOTCHA

You do not need `@Repository` on interfaces extending a Spring Data base interface — they are picked up automatically. Exceptions are still translated into Spring's `DataAccessException` hierarchy.

82 Repository Interface Hierarchy

SPRING DATA

Q What is the difference between Repository, CrudRepository, and JpaRepository?

A They form an inheritance chain, each adding capability. Picking the right one is about exposing only the API you actually want callers to use — a deliberate contract, not just convenience.

`Repository<T, ID>` is a pure marker with no methods; you add only the ones you want. `CrudRepository` adds `save`, `findById`, `findAll`, `delete`, `count`.

`PagingAndSortingRepository` adds paging/sorting overloads. `JpaRepository` is the JPA-specific top of the chain, adding `flush`, `saveAndFlush`, `deleteAllInBatch`, and returning `List` instead of `Iterable`.

In Spring Data 3.x, `CrudRepository` now also extends nothing above `Repository`, and `ListCrudRepository` / `ListPagingAndSortingRepository` exist for List-returning variants. Most apps just use `JpaRepository`; use a narrower interface when you deliberately want to restrict the surface.

```
// Expose only reads – no save/delete leaks to callers
public interface ReadOnlyUserRepo extends Repository<User, Long> {
    Optional<User> findById(Long id);
    List<User> findAll();
}
```

GOTCHA

`JpaRepository` ties you to JPA. If you might swap the store (Mongo, JDBC), depend on `CrudRepository` in your service code to keep it portable.

83 Derived Query Methods

SPRING DATA

Q How do derived query methods (findBy...) work?

A Spring Data parses the method name into a query at startup. The name is split into a subject (`find` , `count` , `exists` , `delete`) and a predicate after `By` . Property names and keywords like `And` , `Or` , `Between` , `LessThan` , `Like` , `OrderBy` , `IgnoreCase` become the WHERE clause and ordering.

Because parsing happens at **bootstrap**, a typo like `findByEmial` fails fast when the context starts, not at call time — a nice safety property. Return types are flexible:

`Optional<T>` , `List<T>` , `Stream<T>` , `Page<T>` , or a projection interface.

The practical limit: once a method name grows past three or four conditions it becomes unreadable. At that point switch to `@Query` , a `Specification` , or QueryDSL. Derived queries shine for simple, stable lookups.

```
List<User> findByLastNameAndActiveTrueOrderByCreatedAtDesc(String last);
Page<User> findByAgeGreaterThan(int age, Pageable page);
Optional<User> findByEmailIgnoreCase(String email);
```

GOTCHA

Nested property traversal is greedy: `findByAddressZipCode` may resolve to `address.zipCode` or `addressZip.code` . Use an underscore — `findByAddress_ZipCode` — to force the split explicitly.

84 The @Query Annotation

SPRING DATA

Q When and how do you use @Query with JPQL and native SQL?

A Use `@Query` when a derived method name would be too complex, when you need joins/aggregation, or when you want an explicit projection. It puts the query text right next to the method so intent is obvious and reviewable.

JPQL is the default and operates on **entities and their fields**, not tables and columns — it is portable across databases. Set `nativeQuery = true` for raw SQL when you need vendor features (window functions, hints) at the cost of portability. Bind parameters positionally (`?1`) or, better, by name with `@Param` .

Modifying statements need `@Modifying` and a surrounding transaction; they bypass the persistence context, so entities already loaded may be stale afterward — consider `clearAutomatically = true` .

```
@Query("SELECT u FROM User u WHERE u.email = :email")
Optional<User> findByEmail(@Param("email") String email);

@Modifying
@Query(value = "UPDATE users SET active = false WHERE id = ?1",
      nativeQuery = true)
int deactivate(Long id);
```

GOTCHA

Native queries returning a subset of columns can't map to the entity directly — use a DTO/interface projection or a constructor expression, or you'll get mapping errors.

85 Entity Mapping & Relationships

SPRING DATA

Q How do you map relationships, and what is the "owning side"?

A In a bidirectional relationship the **owning side** is the one that holds the foreign key and controls what gets written to the database. The inverse side is marked with `mappedBy` and is read-only for persistence purposes. For `@ManyToOne` / `@OneToMany` pairs the many-side (with the FK column) is always the owner.

This is the single most common JPA bug: developers set the collection on the inverse side and expect the FK to update. It won't. You must set the owning reference. Keep both sides in sync with helper methods so the object graph stays consistent in memory.

`@JoinColumn` names the FK column on the owning side. Cascade options (`CascadeType.ALL` , `orphanRemoval=true`) control lifecycle propagation — powerful but easy to over-apply, especially `REMOVE` cascading further than intended.

```
@Entity
class Order {
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<Item> items = new ArrayList<>();
}

@Entity
class Item {
    @ManyToOne // owning side – holds order_id FK
    @JoinColumn(name = "order_id")
    private Order order;
}
```

GOTCHA

Using `@OneToMany` without `mappedBy` makes Hibernate create a join table instead of a FK column. Almost never what you want for a parent-child relationship.

86 N+1 Selects & Fetch Types

SPRING DATA

Q What is the N+1 select problem, and how do LAZY, EAGER, and fetch joins relate?

A N+1 happens when you load N parent entities with one query, then trigger one extra query per parent to fetch a lazy association — 1 + N round trips. It is the classic performance killer in JPA apps and usually invisible in dev with small data.

`FetchType.LAZY` (default for collections and `@OneToMany`) loads the association only on first access via a proxy. `EAGER` (default for `@ManyToOne`) loads it immediately. EAGER does not fix N+1 — it often makes it worse by firing extra selects unconditionally. The real fix is telling JPA to fetch what you need in **one query**.

Use a `JOIN FETCH` in JPQL, or an `@EntityGraph` on the repository method, to load the graph eagerly for that specific query. Keep mappings LAZY and fetch explicitly per use case — the golden rule of JPA performance.

```
@Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.id = :id")
Optional<Order> findWithItems(@Param("id") Long id);

@EntityGraph(attributePaths = "items")
List<Order> findByStatus(String status);
```

GOTCHA

Accessing a LAZY association after the transaction/session closes throws

`LazyInitializationException`. Fetch inside the transaction or map to a DTO before returning — never rely on open-session-in-view.

87 Transactions & Propagation

SPRING DATA

Q How does `@Transactional` work, and what do propagation levels mean?

A `@Transactional` is implemented with an AOP proxy: on entry it begins (or joins) a transaction, on normal return it commits, and on a runtime exception it rolls back. Because it is proxy-based, a **self-invocation** (a method calling another `@Transactional` method on `this`) bypasses the proxy and the annotation is ignored.

Propagation defines how a method behaves relative to an existing transaction.

REQUIRED (default) joins one or creates one. **REQUIRES_NEW** suspends the current and starts an independent one — useful for audit logging that must persist even if the outer call rolls back. **NESTED** uses a savepoint. **MANDATORY** requires an existing transaction, **SUPPORTS** / **NOT_SUPPORTED** / **NEVER** cover the rest.

By default Spring rolls back only on unchecked exceptions and **Error**, not checked exceptions — override with `rollbackFor`. Use `readOnly = true` for query methods to hint the provider and skip dirty-checking overhead.

```
@Transactional(propagation = Propagation.REQUIRES_NEW,
                rollbackFor = Exception.class)
public void writeAuditEntry(AuditEvent e) { ... }
```

GOTCHA

Marking a method `@Transactional` but calling it internally from the same class silently does nothing. Move the call to another bean, or self-inject the proxy.

88 @RestController vs @Controller

SPRING WEB

Q What is the difference between `@RestController`, `@Controller`, and `@ResponseBody`?

A `@Controller` is the classic MVC stereotype: handler methods return a **view name** that a `ViewResolver` renders (Thymeleaf, JSP). The return value is a logical view, not the response body.

`@ResponseBody` flips that: the return value is serialized directly into the HTTP response body via an `HttpMessageConverter` (Jackson for JSON). `@RestController` is simply `@Controller + @ResponseBody` applied to every method — the standard choice for REST/JSON APIs.

So the rule of thumb: use `@Controller` when serving server-rendered HTML pages, and `@RestController` when serving data (JSON/XML) to clients. You can still mix — put `@ResponseBody` on one method of a plain `@Controller`.

```
@RestController
@RequestMapping("/api/users")
public class UserApi {
    @GetMapping("/{id}")
    public UserDto get(@PathVariable Long id) { return service.find(id); }
}
```

GOTCHA

On a plain `@Controller`, returning a `String` without `@ResponseBody` is treated as a view name — you'll get a confusing "view not found" instead of the string in the body.

89 Request Mapping Annotations

SPRING WEB

Q How do @GetMapping, @PostMapping and friends work?

A These are composed shortcuts for `@RequestMapping(method = ...)`.

`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping`

each bind a URL pattern to one HTTP verb, which is exactly what you want for clean, RESTful routing.

You can narrow the match further: `consumes` restricts by request Content-Type, `produces` by Accept header, and `params` / `headers` by presence of specific values. A class-level `@RequestMapping` supplies a common path prefix that method-level mappings extend.

Spring MVC resolves the **most specific** match. Ambiguous mappings (two methods matching the same request) fail at startup or throw at runtime, which surfaces routing mistakes early.

```
@PostMapping(value = "/users",
              consumes = MediaType.APPLICATION_JSON_VALUE,
              produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<UserDto> create(@RequestBody CreateUser req) {
    return
    ResponseEntity.status(HttpStatus.CREATED).body(service.create(req));
}
```

GOTCHA

Spring Boot 3 uses `PathPatternParser` by default; trailing-slash matching is **no longer** automatic. `/users` and `/users/` are now distinct — a common upgrade surprise.

90 @RequestBody & @ResponseBody

SPRING WEB

Q How do @RequestBody and @ResponseBody handle serialization?

A Both are driven by `HttpMessageConverter` s. `@RequestBody` deserializes the raw request body into a Java object; `@ResponseBody` serializes a returned object into the response body. The chosen converter depends on Content-Type / Accept negotiation — for JSON that's `MappingJackson2HttpMessageConverter` .

This means the request body is read **once** and bound to a single parameter — you cannot have two `@RequestBody` params. It also means binding failures (malformed JSON, type mismatch) throw `HttpMessageNotReadableException` before your method runs, which you handle globally.

Combine `@RequestBody` with `@Valid` to trigger bean validation on the deserialized object. For fine-grained control over status and headers, wrap the response in `ResponseEntity` instead of relying on the implicit 200.

```
@PostMapping("/orders")
public OrderDto place(@Valid @RequestBody OrderRequest req) {
    return orderService.place(req);    // serialized back to JSON
}
```

GOTCHA

Jackson needs a way to construct the object. With no default constructor and no `@JsonCreator` /parameter-name info, deserialization fails. For records, Spring/Jackson bind by component names — make sure parameter names are compiled in (`-parameters`).

91 @PathVariable vs @RequestParam

SPRING WEB

Q When do you use @PathVariable versus @RequestParam?

A `@PathVariable` extracts a value that is **part of the URI path** — it identifies a resource, e.g. `/users/42`. `@RequestParam` reads a query-string parameter (or form field), e.g. `/users?active=true&page=2`. The distinction mirrors REST semantics: path = which resource, query = how to filter/shape it.

Both support type conversion and can be made optional. `@RequestParam` offers `required=false` and `defaultValue`. Path variables are inherently required because they are structural. You can bind a `Map` or a `MultiValueMap` to capture all params dynamically.

Use path variables for identifiers and query params for pagination, sorting, filtering, and optional flags. Mixing them up (e.g. an ID as a query param) is a design smell reviewers notice.

```
@GetMapping("/users/{id}/orders")
public List<OrderDto> list(
    @PathVariable Long id,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(required = false) String status) { ... }
```

GOTCHA

If the method parameter name differs from the path token or query key (common after minification/no `-parameters`), binding fails. Name it explicitly:

```
@PathVariable("id") .
```

92 Global Exception Handling

SPRING WEB

Q How do you handle exceptions globally, and what is ProblemDetail?

A Put cross-cutting exception handling in one place with `@RestControllerAdvice`. Methods annotated `@ExceptionHandler(SomeException.class)` intercept exceptions thrown by any controller and map them to a clean response, keeping error logic out of every handler.

Spring 6 / Boot 3 adopt **RFC 7807** via `ProblemDetail` — a standardized error body with `type`, `title`, `status`, `detail`, and `instance` fields plus custom properties. Extend `ResponseBodyExceptionHandler` to get consistent handling of Spring's built-in MVC exceptions as `ProblemDetail` too.

Return a `ProblemDetail` or `ResponseBody<ProblemDetail>` so clients get machine-readable, uniform errors. This beats hand-rolled error maps and interoperates with tooling that understands the standard.

```
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(NotFoundException.class)
    public ProblemDetail handle(NotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Resource not found");
        return pd;
    }
}
```

GOTCHA

An `@ExceptionHandler` for a broad type like `Exception.class` can shadow more specific handlers. Spring picks the closest match, but ordering across multiple advices needs `@Order` to be deterministic.

93 Bean Validation

SPRING WEB

Q How does bean validation work with @Valid and constraint annotations?

A Spring integrates Jakarta Bean Validation (Hibernate Validator). Annotate fields with constraints like `@NotNull`, `@NotBlank`, `@Size`, `@Email`, `@Min`, then put `@Valid` on the controller parameter. Spring validates before your method body runs, so invalid data never reaches business logic.

When validation fails on a `@RequestBody`, Spring throws `MethodArgumentNotValidException`; for `@RequestParam` / `@PathVariable` with `@Validated` at class level it throws `ConstraintViolationException`. Handle both in your `@RestControllerAdvice` to produce a field-level error response, ideally as a `ProblemDetail` with a list of violations.

Use `@Validated` (Spring's variant) to enable **validation groups** — e.g. different rules on create vs update. Nested objects need `@Valid` on the field to cascade validation into them.

```
public record CreateUser(  
    @NotBlank String name,  
    @Email String email,  
    @Min(18) int age) { }  
  
@PostMapping("/users")  
public UserDto create(@Valid @RequestBody CreateUser req) { ... }
```

GOTCHA

Validation on service-layer beans only fires if the class is annotated `@Validated` and invoked through the Spring proxy. Directly instantiated objects are never validated automatically.

94 Spring Security Architecture

SPRING SECURITY

Q How does the Spring Security filter chain work?

A Spring Security is fundamentally a **chain of servlet filters**. A single `DelegatingFilterProxy` hands the request to a `FilterChainProxy`, which selects the matching `SecurityFilterChain` and runs its ordered filters — things like `CsrfFilter`, authentication filters, `AuthorizationFilter`, and `ExceptionTranslationFilter`.

In Security 6 you configure this by declaring a `SecurityFilterChain` *bean* using the lambda DSL — the old `WebSecurityConfigurerAdapter` is removed. Each filter has one job; authorization happens late in the chain via `AuthorizationFilter`, and authentication results are stored in the `SecurityContextHolder`.

Understanding it as filters explains a lot: security runs **before** your controller, exceptions from it become 401/403 (not handled by your `@ControllerAdvice` unless you wire it up), and method security is a separate, later layer via AOP.

```
@Bean
SecurityFilterChain security(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(a -> a
            .requestMatchers("/public/**").permitAll()
            .anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults())
        .build();
}
```

GOTCHA

Filter-chain errors (401/403) bypass MVC exception handling. To return a JSON `ProblemDetail` on auth failure, configure an `AuthenticationEntryPoint` / `AccessDeniedHandler`, not just `@ExceptionHandler`.

95 Authentication vs Authorization

SPRING SECURITY

Q What is the difference between authentication and authorization?

A Authentication answers "who are you?" — verifying identity from credentials (password, token, certificate). **Authorization** answers "what are you allowed to do?" — checking whether the authenticated principal has the required roles/authorities for an action. Authentication always comes first.

In Spring Security, authentication produces an `Authentication` object (principal + granted authorities) stored in the `SecurityContext`. Authorization then consults those authorities. Endpoint-level rules live in `authorizeHttpRequests`; method-level rules use `@PreAuthorize` / `@PostAuthorize` when you enable `@EnableMethodSecurity`.

A subtle distinction: a **role** is just an authority conventionally prefixed `ROLE_`.

`hasRole("ADMIN")` checks for authority `ROLE_ADMIN`; `hasAuthority("ADMIN")` checks the literal string. Mixing these up is a frequent source of "why is my 403 happening?".

```
@PreAuthorize("hasRole('ADMIN') or #id == authentication.name")
public UserDto get(String id) { ... }
```

GOTCHA

401 (Unauthorized) means *not authenticated*; 403 (Forbidden) means *authenticated but not authorized*. The HTTP name for 401 is misleading — it is really about authentication.

Q How do you implement stateless authentication with JWT?

A Stateless means the server keeps **no session**. Each request carries a signed JWT (usually in the **Authorization: Bearer** header) that encodes the identity and authorities. The server validates the signature and expiry on every request — no server-side session store, which scales horizontally cleanly.

Configure the session policy to **STATELESS** and treat the app as an OAuth2 Resource Server. Spring Security's **oauth2ResourceServer().jwt()** validates tokens automatically against a configured issuer/JWK set and populates the **SecurityContext** from the claims — you rarely need a hand-written filter anymore.

The trade-off: JWTs are hard to revoke before expiry. Keep access tokens short-lived and pair them with refresh tokens, or maintain a denylist for forced logout. Never put secrets in the payload — it is signed, not encrypted, and trivially decoded.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .sessionManagement(s -> s.sessionCreationPolicy(
            SessionCreationPolicy.STATELESS))
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .csrf(csrf -> csrf.disable())    // safe: no cookies/session
        .build();
}
```

GOTCHA

Mapping claims to authorities needs a **JwtAuthenticationConverter**. By default Spring expects a **scope** / **scp** claim and prefixes authorities with **SCOPE_**, not **ROLE_** — so **hasRole** checks silently fail.

97 Password Encoding

SPRING SECURITY

Q How does password encoding work with BCrypt and PasswordEncoder?

A Passwords must never be stored in plaintext or with a plain hash. Spring Security's `PasswordEncoder` abstraction hashes with a **salt** and a deliberately slow, adaptive algorithm. `BCryptPasswordEncoder` is the common default; Argon2 and SCrypt are also supported for higher resistance.

BCrypt embeds the salt and a cost factor (work factor, default 10) inside the resulting hash string, so verification needs no separate salt column. The cost makes brute-forcing expensive and can be raised as hardware improves. `matches(raw, encoded)` re-hashes and compares in constant time.

Prefer `PasswordEncoderFactories.createDelegatingPasswordEncoder()`, which prefixes hashes with an id like `{bcrypt}`. This lets you **migrate algorithms** over time — old hashes keep verifying while new ones use the upgraded scheme.

```
@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}
// stored: {bcrypt}$2a$10$N9qo8uL... - id + salt + cost baked in
```

GOTCHA

Never compare passwords with `==` or `equals` on hashes, and never call the encoder yourself for verification — always use `matches()` so comparison is timing-safe and the embedded salt/cost are honored.

98 CORS and CSRF

SPRING SECURITY

Q How do CORS and CSRF differ, and how do you configure them?

A They solve opposite problems. **CORS** (Cross-Origin Resource Sharing) is a browser mechanism that *relaxes* the same-origin policy so a JS app on one origin may call your API on another — you opt in by returning the right `Access-Control-Allow-*` headers.

CSRF (Cross-Site Request Forgery) is an *attack* where a malicious site tricks a browser into sending an authenticated request using the victim's cookies.

Configure CORS via a `CorsConfigurationSource` and enable it in the filter chain (`http.cors(...)`) — do not just slap `@CrossOrigin` everywhere. CSRF protection is on by default and matters when you authenticate with **cookies/sessions**; Spring issues a token that must accompany state-changing requests.

Key rule: for a stateless, token-in-header API (no cookies), CSRF is not applicable and is commonly disabled. For a cookie-based session app, keep CSRF enabled and use the `CookieCsrfTokenRepository` pattern for SPAs.

```
http
    .cors(Customizer.withDefaults())
    .csrf(csrf -> csrf.csrfTokenRepository(
        CookieCsrfTokenRepository.withHttpOnlyFalse()));
```

GOTCHA

Disabling CSRF because "it blocks my POSTs" is dangerous if you use session cookies — you reopen the CSRF hole. Only disable it when auth is genuinely stateless (bearer tokens, no cookies).

99 Testing Spring Applications

SPRING TESTING

Q What is the difference between `@SpringBootTest`, `@WebMvcTest`, and slices?

A `@SpringBootTest` loads the **full application context** — an integration test. It is thorough but slow, and pairs with `MockMvc` (via `@AutoConfigureMockMvc`) or a real port (`webEnvironment=RANDOM_PORT`) plus `TestRestTemplate`.

Slice tests load only part of the context for speed and focus. `@WebMvcTest` loads the web layer (controllers, filters, Jackson) but not services or repositories — you provide those as mocks. `@DataJpaTest` loads JPA/repository beans with an in-memory DB and a rolled-back transaction per test. `@JsonTest`, `@RestClientTest` cover narrower slices.

For collaborators, register mocks in the context. In current Boot use `@MockitoBean` (Boot 3.4+; the older `@MockBean` is deprecated) to replace a bean with a Mockito mock. Combine with plain Mockito `when/verify` for behavior. Choose the narrowest slice that still exercises what you're testing.

```
@WebMvcTest(UserApi.class)
class UserApiTest {
    @Autowired MockMvc mvc;
    @MockitoBean UserService service; // replaces the real bean

    @Test void returnsUser() throws Exception {
        when(service.find(1L)).thenReturn(new UserDto(1L, "Al"));
        mvc.perform(get("/api/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Al"));
    }
}
```

GOTCHA

`@WebMvcTest` does **not** load your `SecurityFilterChain` service beans, but it does apply security auto-config. Unmocked security can cause unexpected 401s — import your security config or use `@WithMockUser`.

100 Microservices with Spring Cloud

SPRING CLOUD

Q How do you build microservices with Spring Cloud (discovery, config, gateway, resilience)?

A Spring Cloud provides the cross-cutting infrastructure that microservices need so each service can focus on business logic. The four pillars: **service discovery**, **centralized configuration**, an **API gateway**, and **resilience** patterns.

Service discovery (Eureka, or Kubernetes-native) lets services find each other by logical name instead of hardcoded hosts, enabling client-side load balancing via `Spring Cloud LoadBalancer`. Spring Cloud Config Server externalizes configuration (often git-backed) so all instances share consistent, versioned settings and can refresh without redeploy. `Spring Cloud Gateway` is the single entry point handling routing, auth, rate limiting, and header rewriting.

Resilience comes from `Resilience4j` (Netflix Hystrix is retired): **circuit breakers** stop cascading failures by short-circuiting a failing downstream call and invoking a fallback, plus retries, bulkheads, and rate limiters. Together these keep a partial outage from taking down the whole system.

```
@CircuitBreaker(name = "inventory", fallbackMethod = "fallback")
public Stock check(String sku) {
    return inventoryClient.getStock(sku);
}
public Stock fallback(String sku, Throwable ex) {
    return Stock.unknown(sku);    // degrade gracefully
}
```

GOTCHA

A circuit breaker without a sensible fallback just converts one failure into another. Also, retries stacked on a circuit breaker can amplify load — order and configure them deliberately (retry inside, breaker outside).

You made it to 100.

If you can explain most of these out loud, you're ready. The candidates who stand out aren't the ones who memorized answers — they're the ones who can reason about trade-offs, so when you hit a question that isn't here, walk through it the same way: what problem does this solve, and what does it cost?

Found this useful?

This is part of a series of deep-dive interview guides. Follow for the next one — a new technology every few days.

Save this PDF and share it with someone who's interviewing.